

Agent-Centric Data System: Architecture, Literature, and Challenges

Zuohan Wu^{*†} · Pengze Chen^{*‡} · Xinyi Zhu^{*†} · Hao Wang^{*†} · Jingbo Qiao^{*†} · Jingyi Zou^{*†} · Liping Wang^{*†} · Zitong Qu^{*†} · Xujia Li^{*‡} · Lei Chen^{†‡}

Abstract Agentic AI relies on data systems to access information, retain task state, and carry out operations. In many applications, agents also operate within the data-system stack, where they interpret requests and coordinate work over stored data and external services. Because later steps depend on intermediate results, this work evolves during execution rather than arriving as a fixed query. Relevant research spans database systems, AI agents, information retrieval, and large language model serving. This survey presents an Agent-Centric Data System (ACDS) architecture to organize these studies. ACDS separates the agent task flow into Perception, Planning, Execution, and Action, supported by Memory, Data Interface, and Inference Infrastructure. For each layer, we define its role, inputs, and outputs, then review representative systems according to the data-management function they provide. The review covers how agents interpret requests, construct and realize plans, interact with tools and data, retain workflow state, and use distributed compute and storage. Across the architecture, plan changes can invalidate task state, external actions complicate authorization and recovery, and execution depends on coordinated data and inference resources. The review identifies current support and open data-system problems across the agent workflow.

Keywords Agentic AI · Data Systems · Multi-Agent Systems

1 Introduction

A data system is a software stack that manages the storage, retrieval, organization, and processing of data, serving as the interface between data producers and data consumers [59, 79]. Traditional data systems commonly receive explicit user or application requests and process them through established query or application pipelines. Agentic AI changes the requirements placed on data systems in five ways:

- **Semantic Interfaces.** Agents often begin with a domain-level request rather than a fully specified query. Natural-language access therefore extends beyond direct Text-to-SQL translation [5]. Interpretation may also require data relationships, provenance, and freshness.
- **Repeated and Overlapping Requests.** Agents inspect schemas, sample records, and test partial solutions before completing a task. Repeated probes across steps or agents can share computation and results [59].
- **Persistent Task State.** Later steps depend on earlier observations, plans, and actions [83, 118]. The corresponding task record may span several model calls or agents and may also be needed to resume work after a failure.
- **Cross-System Actions.** A task can interleave database operations with calls to external tools and services [85, 131]. Effects that span systems generally cannot be rolled back by one database transaction.
- **Iterative Execution.** Intermediate results can change what the agent does next, while execution feedback can also be used to refine the workflow [107, 141]. The workload therefore develops during execution.

Existing data systems commonly serve users or applications with relatively stable information needs. Agent workflows also explore, retain state, act, and verify across multiple steps. To address this gap, we present **ACDS Architecture** (Agent-centric Data System Architecture), a reference architecture that organizes the data-system functions used by

^{*}These authors contributed equally to this work.

[†]The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

[‡]The Hong Kong University of Science and Technology, Hong Kong SAR, China

Xujia Li (corresponding author)

E-mail: leexujia@ust.hk

these workflows. ACDS follows requests from perception and workload planning through agent execution and external action, with Memory, the Data Interface, and Inference Infrastructure supporting the task across steps.

Figure 1 maps the agentic requirements above to the seven ACDS layers. **To provide semantic interfaces**, the Perception Layer interprets user inputs, while the Data Interface Layer exposes heterogeneous resources as unified DataObjects. **To handle redundant probes**, the Planning Layer consolidates similar requests across the workload, and the Execution Layer assigns the tasks to agents [104]. **To govern diverse actions**, the Action Layer resolves tool bindings and records their effects. The Memory Layer **supports stateful workflows** by retaining records across steps and agents [122]. Meanwhile, the Inference Infrastructure Layer provides the runtime substrate. **To support iterative discovery**, execution outcomes return to the Planning Layer for replanning and to the Memory Layer for use in later steps. Using this architecture to organize the review, Sects. 2–8 define each layer’s inputs, outputs, and core functions, then examine related work and open problems. After the layer-by-layer review, Sect. 9 brings together the challenges shared by several layers and summarizes their relationship in Table 8.

This survey makes three contributions:

- We introduce the ACDS reference architecture, which relates the task flow of an agent to the data-system functions that support it.
- We use the ACDS layers as a taxonomy for categorizing existing approaches and comparing the data-system functions they provide.
- We identify open challenges and research directions for agent-centric data systems.

1.1 ACDS Definitions

Figure 1 summarizes the scope of ACDS and the objects exchanged across its layers. The main task path contains four transformations:

$$\begin{aligned} Q &= \text{Perception}(\mathcal{E}, \Sigma), \\ \text{SQ} &= \text{Planning}(Q, \Sigma), \\ \Pi_E &= \text{Execution}(\text{SQ}, \Sigma), \\ Z &= \text{Action}(\Pi_E, \Sigma). \end{aligned} \tag{1}$$

Equation (1) names the information passed between these functions. An implementation may combine several functions and may return to an earlier one when new observations or failures require revision. Here, $\mathcal{E}$, Q , SQ , Π_E , and Z denote task events, perceived queries, scheduled queries, an agent execution plan, and outcome records, respectively. A task event may contain one request, a batch of requests, or the newly arrived part of a stream; accordingly, Q may describe

either a single request or a workload. The event $\mathcal{E}$ enters from a user or application, and each subsequent first argument is the output of the preceding layer. The second argument, Σ , supplies state shared across the task. We define this shared system state as

$$\Sigma = \langle M, D, I \rangle, \tag{2}$$

where M is the Memory View, D is the Data View, and I is the Runtime Realization. The three components connect task state, persistent data, and physical resources. Memory constructs M from task records persisted and indexed through the Data Interface. These records retain references to the source-backed DataObjects represented in D . The Data View covers both external resources $\mathcal{R}$ and stores maintained by ACDS for memory records and intermediate artifacts. The Runtime Realization I records the placement of active model calls, data operations, and managed state across compute, storage, and communication resources.

The task path and Σ evolve together. Records derived from Q , SQ , Π_E , and Z may be retained by Memory and persisted through the Data Interface. A verified resource change in Z refreshes the corresponding entries in D . Model calls, data operations, and managed state update I .

Perception Layer. Perception resolves an incoming event against the state available when it arrives. M supplies information from earlier steps or interactions, and D supplies a referenced document, report, schema, or other data object. The output Q contains the interpreted request and references to the state on which that interpretation depends.

Planning Layer. Planning processes Q as a workload. It identifies logical units and their relations, allowing compatible work to be shared while retaining the required order and isolation. The resulting SQ covers every perceived query represented in Q . A shared operation also retains a route from its result to each query that it serves.

Execution Layer. Execution chooses how agents realize SQ . It maps the scheduled work to agent invocations and records their assignments, dependencies, and execution stages in Π_E . Current runtime conditions in I may influence this arrangement. Verification or recovery branches may also appear in the plan. Action uses Π_E for external operations, while Inference Infrastructure schedules its runtime work.

Action Layer. Action applies Π_E to the external capabilities exposed through D , using M when an operation depends on earlier task state. It resolves and validates each operation, executes it, and checks the returned observation or state change. The output Z records the outcome and the evidence needed to trace it. All changes to external state pass through Action.

Memory Layer. Memory determines the lifecycle and relationships of task records and selects the authorized records that form M . Records chosen for retention are handed to the Data Interface for persistence and indexing. Stored records

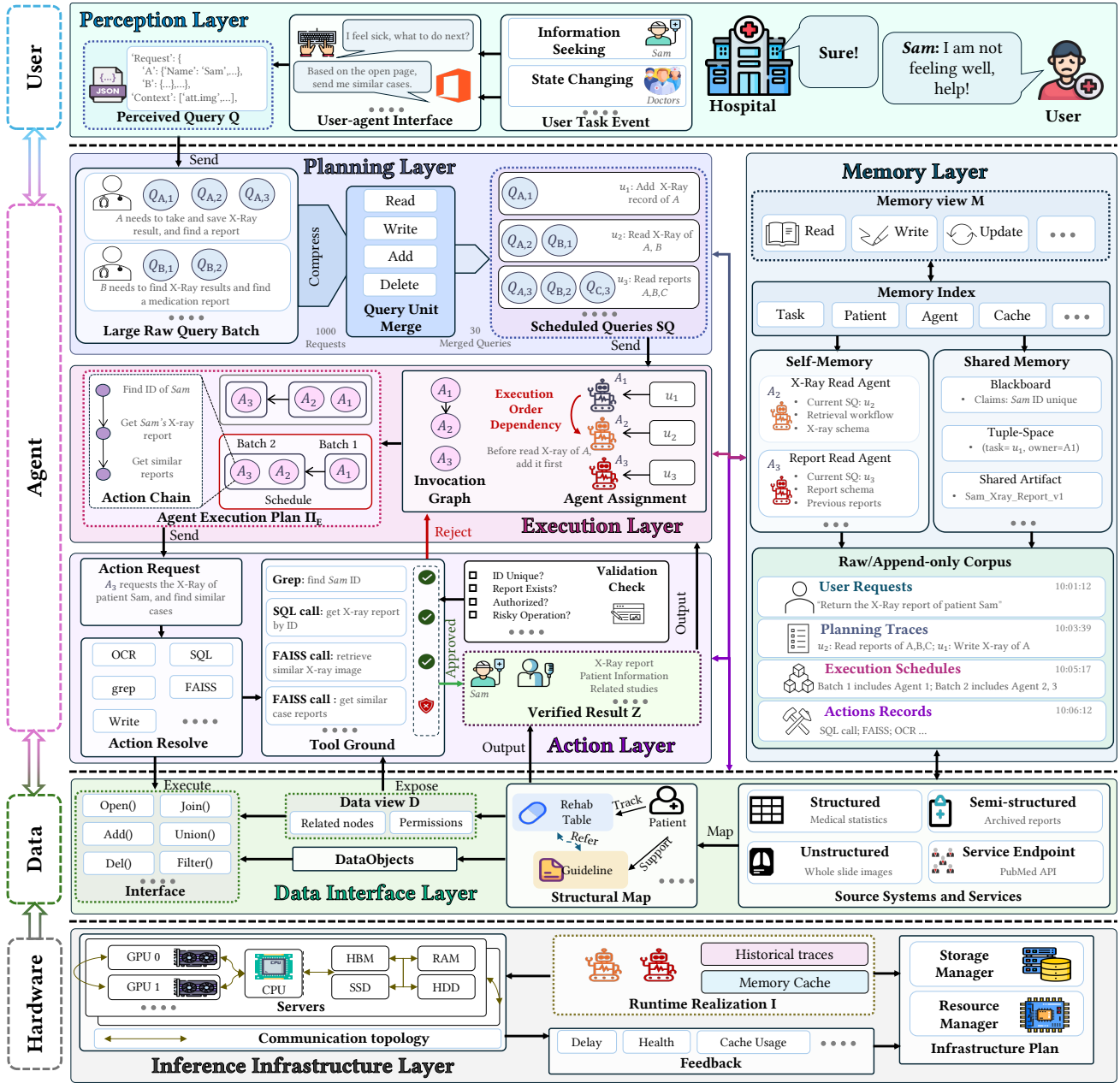


Fig. 1: Overview of ACDS and the interfaces among its seven layers. Perception, Planning, Execution, and Action transform an incoming task event $\mathcal{E}$ into Q , SQ , Π_E , and Z . Memory, the Data Interface, and Inference Infrastructure supply M , D , and I , which connect task history, source-backed data, and physical execution to this task path. The hospital example shows how a batch of requests moves through these interfaces

remain linked to DataObjects in D , so later access can detect changes in source data.

Data Interface Layer. The Data Interface maintains D over external resources and stores managed by ACDS. It presents source data and system records as source-backed DataObjects and exposes the operations supported by their underlying systems. The view preserves source query and update semantics and connects each result to its source. Memory

sends retained task records to this layer for persistence and indexing, then uses the returned handles when constructing M . Action accesses or modifies resources through D , and verified outcomes refresh affected objects.

Inference Infrastructure Layer. Inference Infrastructure maintains I for the model and data-processing work issued across the task-processing layers, Memory, and the ACDS-managed stores represented in D . It maps model calls and

supporting data operations to workers and devices under Π_E 's assignments and dependencies. Its storage scheduler distributes persistent agent state and transient model state across storage nodes and memory tiers, moving or replicating them as demand and data locality change. Communication scheduling coordinates transfers between workers and storage nodes.

Running example. Throughout the survey, we use the hospital workload in Figure 1. The perceived query Q contains requests for Patients A , B , and C , where Patient A is Sam. Patient A 's request contributes an X-ray write, an X-ray read, and a report read. Patient B requests an X-ray and a report, while Patient C requests a report. Planning keeps the write as u_1 , combines compatible X-ray reads in u_2 , and combines report reads in u_3 . The precedence $u_1 \prec u_2$ preserves the write-before-read requirement. Execution realizes these units as invocations v_1, v_2, v_3 , assigns them to agents a_1, a_2, a_3 , and places independent invocations in the same stage. Action carries out the corresponding data operations and records their outcomes. Memory retains the task records, the Data Interface exposes the referenced patient, X-ray, and report objects through D , and Inference Infrastructure realizes the required physical work in I .

1.2 Survey Scope

This paper covers systems and research related to the seven layers in Sect. 1.1. We prioritize recent and representative work that implements, analyzes, or evaluates an ACDS function. The scope crosses several research communities within computer science. Across Sects. 2–8, 115 sources receive substantive discussion after works cited in more than one layer are deduplicated. Of these, 105 academic works span 11 arXiv CS categories. The largest groups are computation and language (31 works), databases (27), and artificial intelligence (18), while the remaining 29 are distributed across eight smaller areas, led by distributed computing and multi-agent systems. The corpus also includes eight deployed systems and two technical specifications. Table 1 compares its scope with three representative surveys and two architecture papers. Existing surveys organize the literature around general agent construction, data agents, or agentic AI. This survey instead follows the data-system functions that support an agent workflow and defines the interfaces between them. System papers such as the Agent-First Data System [59] and Proactive Data Systems [137] propose related architectural directions but do not review work across all seven layers or define the same layer inputs, outputs, and shared state.

2 Perception Layer

The Perception Layer performs the first transformation in Eq. (1). It interprets a task event $\mathcal{E}$ against the relevant shared state Σ and produces the perceived query Q used by Planning. This section examines how the interaction surface determines the context available to Perception and when that context changes.

Running example. In Figure 1, $\mathcal{E}$ contains hospital requests concerning Patients A , B , and C . For Patient A , Perception connects the request to Sam's patient record and the newly acquired X-ray. The resulting Q retains these references together with the interpreted requests that Planning will process.

2.1 Layer Model

S denotes the surface through which $\mathcal{E}$ enters the system. The event specifies the intended effect and therefore determines what Q identifies. An information-seeking event needs evidence, while a state-changing event also identifies the operation and target. Sustained assistance draws on prior task state.

The surface also determines which context is available and when it changes. Conversation provides dialogue and attachments, whereas embedded work also supplies the active artifact. Environment-control agents observe the interface again after acting, while background delegation retains state between sessions. Thus, Q contains the interpreted request and references to the context available at that point. We organize the review by S and use event types for comparison.

2.2 Existing Work

Conversational command. In conversation, Q begins with the user's utterance and adds the context required by the requested effect. For database questions, DB-GPT combines natural-language interaction with retrieved knowledge and database access [127]. Mirror makes its generated SQL inspectable and editable before presenting the result [126]. Enterprise assistants ground questions in curated semantics. Databricks Genie uses a Genie Space backed by Unity Catalog and domain instructions [13], while Snowflake Cortex Analyst uses semantic views of business concepts and relationships [109]. The same surface can initiate state changes. Codex CLI can modify files in a local repository and run installed tools [77]; SWE-agent provides a repository interface for editing and testing [129]. For sustained interaction, Generative Agents retrieves earlier observations from a memory stream [83]. Across these cases, the requested effect determines which task state accompanies the utterance.

Table 1: Comparison of ACDS with related surveys and architecture papers. ✓ = directly addressed; ○ = partially or tangentially addressed; blank = not addressed.

Dimension	ACDS Architecture in 2026	Autonomous Agents Survey in 2024 [118]	Data Agents Survey in 2025 [147]	Agentic AI Survey in 2026 [2]	Agent-First Database in 2026 [59]	Proactive Data System in 2025 [137]
Scope and type						
End-to-end architecture	✓				✓	✓
Literature review	✓	✓	✓	✓		
Formal layer definitions	✓				✓	
Task Planning	✓	○		○	✓	✓
Agent Execution	✓	○	✓	✓		
Memory and State	✓	○				
Data Interface and Access	✓				○	○
Inference Infrastructure	✓					

Embedded work. An embedded agent inherits the active object from its host application. Power BI Copilot answers questions using the current report and its semantic model [67]. NL4DV translates a request over a table into a JSON analytic specification and candidate Vega-Lite charts [73]. In a coding environment, GitHub Copilot uses the code visible in the editor and related workspace context, binding Q to the active artifact [32].

Environment control. An action can invalidate the observation on which Q was built. The Mind2Web baseline filters raw HTML before an LLM selects the target element and action [15]. WebArena provides self-hosted websites for multi-step tasks [146], while OSWorld extends evaluation to desktop workflows that span applications [123]. After each action, Q is rebuilt from the new observation and action history.

Background delegation. A delegated run continues after the initiating session, so its progress becomes part of later Q . OpenAI Deep Research returns a source-cited report after multi-step research [76]. Data Interpreter updates a task graph from execution results [38]. Codex Cloud returns a summary and diff from a cloud repository task [78]. A later Q therefore combines the original request with this accumulated state.

Conversation and embedded work rebuild Q each turn. Environment control refreshes it after actions, whereas background delegation retains it across sessions.

2.3 Challenges and Future Directions

As Q follows a task, its context can lose its source or become stale. Either problem makes failures difficult to diagnose.

Keeping source information in Q . Moving a task between surfaces can turn a live artifact into copied text. A background worker may receive file contents without knowing which file the user selected or whether it remains accessible. The difficulty lies in preserving the origins and access conditions of user instructions, retrieved documents, and interface

observations within one Q . MCP provides a common way to expose resources and tools [40], while Perception still decides which of them enter Q and how their roles are recorded. One direction is to retain resource handles and provenance links in Q , following database provenance in relating derived results to their source data [6]. The receiving surface can then resolve the handle and check access before reading the artifact. Marking external text as evidence also gives downstream layers a basis for rejecting instructions found inside it, which matters because indirect prompt injection places malicious instructions in content read by an agent [16].

Updating Q after state changes. An action can invalidate a page observation, while a long-running task may outlive the data from which Q was built. Reconstructing Q after every event retrieves and encodes unchanged material, but a local update requires knowing which parts depended on the changed object. Incremental view maintenance shows how a database can update a derived view from changes to its inputs [35]. The contents of Q are harder to maintain because they are selected according to the task and need not follow a fixed query plan. A perception system could record which source observation supports each selected item, invalidate affected items after a confirmed change, and refresh them before Planning runs again. When those dependencies are uncertain, reconstructing Q avoids carrying stale context forward.

Evaluating the construction of Q . End-to-end success cannot distinguish evidence omitted by Perception from evidence ignored later. There is no single correct Q , since several sets of evidence may support the same task. Evaluation can run that task against the same source state with a fixed planner and executor. A benchmark can record the required evidence and inaccessible material, then change one source and test whether the corresponding part of Q is refreshed. Comparing the resulting executions isolates perception errors without assuming one gold serialization.

3 Planning Layer

The Planning Layer performs the second transformation in Eq. (1). It organizes the workload in Q into logical units and records their relations. Planning then selects the scheduled query SQ sent to Execution. This section follows the construction, rewriting, and selection of these units. Figure 2 summarizes this process.

Running example. Patient A 's request contributes an X-ray write, an X-ray read, and a report read. Patient B contributes an X-ray read and a report read, while Patient C contributes a report read. Planning keeps the write as u_1 , combines the X-ray reads for Patients A and B in u_2 , and combines the report reads for Patients A , B , and C in u_3 . It records $u_1 \prec u_2$ because storing Sam's X-ray precedes reading it.

3.1 Layer Model

Within the Planning Layer, the requests represented in Q are treated as a workload. The layer decomposes the workload into logical units, identifies dependencies and opportunities to share work, and rewrites compatible units without violating their constraints. Runtime feedback may trigger replanning.

The resulting scheduled query is written as

$$SQ = \langle \mathcal{U}, \prec \rangle,$$

where $\mathcal{U}$ is the complete set of logical units passed to the Execution Layer, and $\prec$ is the precedence relation preserved during execution. Each unit contains its logical operation and the planning annotations needed by later layers. These annotations link the unit to the requests it serves and record the constraints that later layers have to preserve, including permissions, isolation requirements, and whether an operation reads or changes external state. The units in $\mathcal{U}$ collectively cover all requests represented in Q . Request links route the result of a merged or shared unit to every request it serves.

3.2 Existing Work

We group the reviewed methods by four capabilities for constructing, selecting, and revising scheduled queries. Table 2 places each method under its primary technical focus and compares corresponding planning capabilities.

Workload representation and unit construction. This category places related requests in a common planning scope and expresses their processing as explicit logical units. Helium represents batches of agentic workflows as query plans and exposes reuse across workflows to the system [113]. Database systems make this idea concrete. SharedDB batches concurrent queries and updates for shared computation [30]. NiagaraCQ considers continuous queries, splitting them and incrementally placing a new query in an existing group when

their expressions permit shared evaluation [8]. This form of reuse assumes that the shared computation has the same result for every participating query. Agent workloads also contain tool calls with external effects, which cannot be reused like repeated reads. LOTUS adds semantic relational operators, while TAG separates natural-language database processing into query synthesis, execution, and answer generation [5, 84]. These representations make the workload and its candidate units inspectable before scheduling.

Relation construction. Relation construction identifies which units depend on one another and which may share work. Required execution ordering is retained in $\prec$, while constraints that prevent sharing remain attached to the affected units. LLMCompiler constructs function-call tasks with a dependency graph that separates parallel from dependent calls [47]. HuggingGPT likewise records task dependencies and execution order [106], while Helium represents workflow DAGs and cross-workflow reuse [113]. These works expose precedence and sharing before merging and scheduling.

Unit optimization and constraints. This category rewrites related units for sharing while leaving incompatible operations apart. Helium prunes dead or duplicate computation and reuses work across workflows, while UQE orders and fuses query clauses, including compatible extractions from the same source [12, 113]. Palimpzest and DocETL use physical-plan search or agentic plan rewriting [58, 104]. Other systems construct or rewrite semantic processing pipelines [84, 101, 105, 114, 115]. BridgeScope aligns tool use with data privileges so that infeasible plans can be identified early [121]. Semantic Integrity Constraints (SICs) add semantic constraints to logical and physical optimization [53].

Selection and replanning. Plan selection compares candidate plans, whereas replanning revises a selected plan after execution results or source changes invalidate an earlier decision. Eddies addresses such runtime change in adaptive query processing by changing operator routing as new information arrives [4]. In agent workloads, the same feedback may also require rebuilding logical units or revisiting earlier sharing, not only changing their execution order. Palimpzest selects physical plans, Abacus searches alternative implementations, and DocETL evaluates and rewrites document-processing plans [58, 96, 104]. AFlow revises workflows using execution feedback. LATS combines tree search with external feedback, while RAFA alternates planning and execution as new feedback arrives [60, 141, 145].

3.3 Challenges and Future Directions

Planning has to decide which requests can share work before Execution chooses how that work will run. It may later have to revisit this decision after some units have completed.

Finding shareable work across requests. SharedDB and NiagaraCQ group queries around common computation [8, 30].

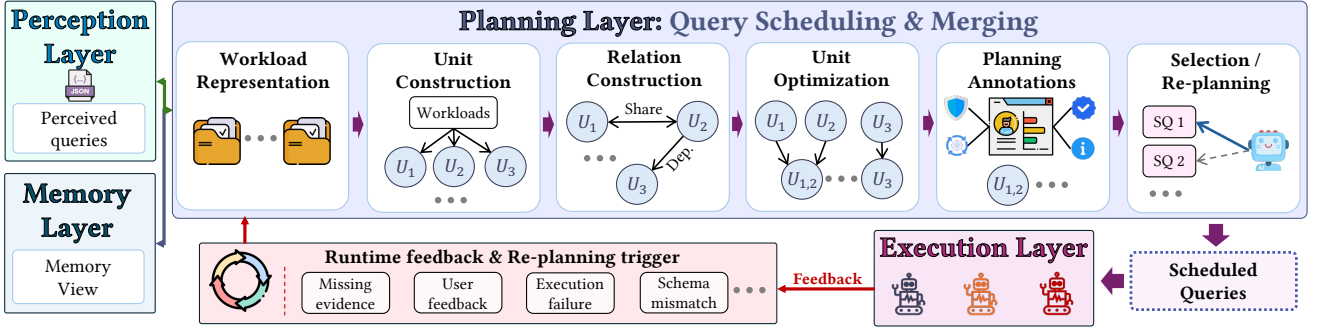


Fig. 2: Planning workflow from Q to SQ . Requests first enter a common workload. Planning constructs logical units and their relations, rewrites compatible units under the planning constraints, and selects a scheduled query. Changes in task state or downstream feedback can start another planning round

Relational queries have algebraic rules for establishing equivalence, whereas natural-language requests do not. Two requests may read the same DataObject but expose different rows to their users. State-changing calls also remain separate even when their arguments look identical. Planning can first share deterministic reads over the same authorized view. A semantic operation can be merged only when one result satisfies every linked request. BridgeScope supplies privilege constraints, and SICs attaches semantic constraints to optimization [53, 121]. Thus, the X-ray reads in the running example can be batched only after access checks determine which rows each request may receive.

Estimation before physical realization. The same workload may admit different unit boundaries and orders. Merging units can reduce calls, but a longer input or lost parallelism may increase latency. Planning cannot know that latency before Execution assigns agents and stages. Palimpzest and Abacus use cost-based search, while DocETL evaluates rewritten document-processing plans [58, 96, 104]. Planning could ask Execution and Infrastructure to estimate a few candidate SQ , then choose their logical units and order. Measurements from completed runs would update later estimates.

Revision after partial execution. Feedback may invalidate a merge or ordering after some units finish. Rebuilding the whole schedule can repeat an external effect or lose the links from a shared result to the requests it serves. A timeout alone does not tell Planning whether a state-changing call took effect. Planning therefore relies on Action to record the outcome before repairing the affected units and precedence edges, as in Eddies [4]. Completed writes remain fixed. In the running example, if the X-ray write has committed, the revised SQ reschedules only its dependent reads.

4 Execution Layer

The Execution Layer performs the third transformation in Eq. (1). It turns SQ into the agent execution plan Π_E . Sched-

uled units become invocations and are assigned to agents. Execution orders these invocations while preserving the logical dependencies chosen by Planning. This section describes their construction, communication, and staging. Figure 3 summarizes this process.

Running example. Execution realizes u_1 , u_2 , and u_3 as invocations v_1 , v_2 , and v_3 . It assigns them to agents a_1 , a_2 , and a_3 . The dependency $u_1 \prec u_2$ requires v_1 to finish before v_2 . Execution therefore places v_1 in the first stage and allows the independent v_3 to run with v_2 in the next stage.

4.1 Layer Model

Let $\mathcal{U}(SQ) = \{u_1, \dots, u_n\}$ denote the scheduled units in SQ , and let $u_i \prec u_j$ mean that u_j requires the result of u_i . We describe an execution plan using four components:

$$\Pi_E = \langle G_E, \mu_E, \lambda_E, \sigma_E \rangle.$$

The invocation graph $G_E = (\mathcal{V}_E, \mathcal{L}_E)$ contains agent invocations as nodes. Its links represent either execution dependencies or communication. A dependency link means that one invocation requires the result of another and therefore imposes an execution order. A communication link imposes order only if its message is required before the receiver starts.

The assignment μ_E associates each invocation with an available and compatible agent. The realization specification λ_E records which invocations realize each scheduled unit and, when several invocations are involved, how their outputs are combined. Associating several units with one invocation represents grouping or fusion, which can reuse shared context. Associating one unit with several invocations may represent either splitting or replication. Split outputs are aggregated, whereas replicated outputs pass an agreement or verifier-based consistency check before the logical result is released. Each invocation may also be expanded into an action chain without introducing a new scheduled unit.

Table 2: Overview of representative planning-layer works grouped by dominant capability.

Category	Method	Unit	Prec.	Share	Constr.	Sel.	Replan	Advantages	Disadvantages
Workload representation and unit construction	Helium [113]	✓	✓	✓					
	SharedDB [30]			✓					
	NiagaraCQ [8]	✓		✓				Creates a joint planning scope and explicit units, enabling reuse across requests.	Conditions for sharing work across users remain weakly modeled.
	LOTUS [84]	✓			✓				
	TAG [5]	✓							
Relation construction	LLMCompiler [47]	✓	✓				✓	Makes dependencies and sharing opportunities explicit.	Few systems state the conditions under which requests from different users or write operations remain separate.
	HuggingGPT [106]	✓	✓						
Unit optimization and constraints	Palimpzest [58]	✓	✓		✓	✓			
	DocETL [104]	✓	✓		✓	✓	✓		
	Unify [114]	✓	✓			✓	✓		
	UQE [12]	✓	✓	✓		✓		Merges or rewrites units while retaining their attached constraints.	Few methods consolidate requests while keeping writes and permission-sensitive operations separate.
	AOP [115]	✓	✓			✓	✓		
	Galois [101]		✓			✓			
	Task Cascades [105]	✓	✓	✓	✓	✓	✓		
	BridgeScope [121]				✓				
	SICs [53]				✓				
Selection and replanning	Abacus [96]		✓		✓	✓			
	Eddies [4]						✓		
	AFlow [141]	✓	✓			✓	✓	Selects candidate plans and revises them using runtime feedback.	Few systems replan a shared workload after an external write has committed.
	LATS [145]					✓	✓		
	RAFA [60]					✓	✓		

Notes. Unit = explicit query or operation-unit construction; Prec. = dependency, precedence, or logical-order planning; Share = common-work elimination or reuse; Constr. = explicit planning constraints; Sel. = complete candidate-plan search and selection; Replan = feedback-driven plan revision. A checkmark indicates direct support for the corresponding capability.

The stage assignment σ_E specifies when each invocation may execute. Invocations in the same stage may run concurrently when dependencies and resources permit. If invocation v_i provides a required result to v_j , then their stages satisfy $\sigma_E(v_i) < \sigma_E(v_j)$. Communication alone imposes no order.

A valid execution plan covers every scheduled unit, assigns every invocation to a compatible agent, and preserves the dependencies in SQ. A dependency may be enforced internally when two units are grouped into one invocation, or externally through a dependency between their invocations. A split or replicated result cannot be used downstream until its aggregation or consistency check is complete. This model describes the common structure of execution plans without assuming that all systems optimize the same objective.

4.2 Existing Work

The model above provides a common vocabulary for comparing execution systems rather than defining a single optimization problem. Existing work can be organized around five execution decisions: unit realization and coverage, agent assignment, invocation-graph construction, runtime staging, and plan selection or recovery. These decisions correspond to λ_E , μ_E , G_E , σ_E , and the overall plan Π_E , respectively.

Unit realization and coverage. The realization specification λ_E determines how scheduled units become concrete model, tool, or agent invocations. HuggingGPT decomposes user requests, selects suitable models, and executes the resulting subtasks [106]. ReWOO separates an upfront reasoning plan from subsequent tool observations and worker

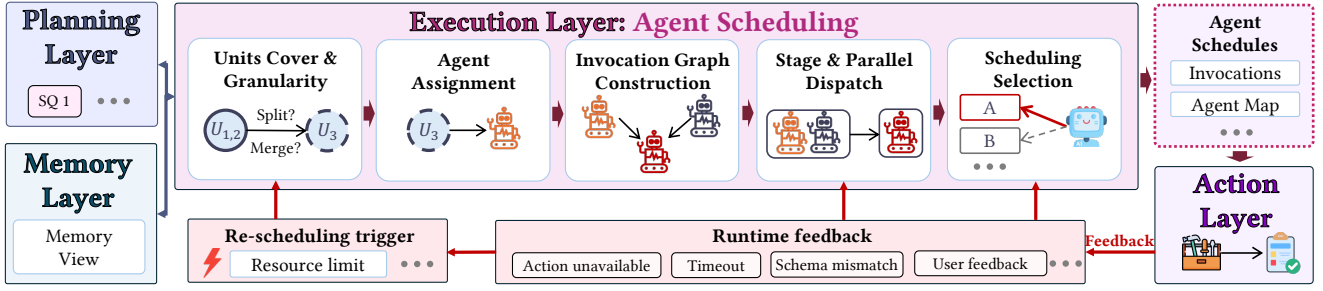


Fig. 3: Execution workflow from SQ to Π_E . Scheduled units become agent invocations, which are assigned to agents and connected by execution or communication links. Execution then groups ready invocations into stages. Runtime feedback may revise the plan, while the dependencies in SQ remain in force

execution [125]. LLMCompiler formulates execution plans for function calls and dispatches independent calls concurrently [47]. AFlow represents workflows as code with LLM-invoking nodes and searches this space using execution feedback [141]. These methods expose different realizations but leave unit coverage and the handling of split or replicated outputs unspecified.

Agent assignment. The assignment μ_E determines which agent handles each invocation. CAMEL uses role-playing to structure cooperation, while AutoGen provides customizable conversable agents and programmable interaction patterns [55, 122]. MetaGPT and ChatDev organize software-development tasks around specialized roles and structured workflows [39, 88]. AgentVerse can adjust the composition of an expert group, and Magentic-One uses an orchestrator to direct specialized agents and track task progress [9, 22]. Their assignments are often predefined, task-specific, or constrained by framework-level roles.

Invocation-graph construction. The graph G_E determines how invocations depend on and communicate with one another. Dryad offers an earlier dataflow example in which vertices run sequential programs and edges carry data [42]. Agent graphs extend this structure with role-specific invocations and optional communication. AutoGen supports flexible conversation patterns, while MetaGPT, ChatDev, and AgentVerse instantiate more structured interaction processes [9, 39, 88, 122]. MacNet organizes agents as DAGs and propagates their interactions in topological order [89]. GPTSwarm represents agents as computational graphs and optimizes graph connectivity, whereas AgentPrune removes redundant or potentially harmful communication [139, 148]. Execution dependencies are not interchangeable with optional communication. Optional messages can be pruned, but dependencies required by SQ cannot.

Runtime staging. The stage assignment σ_E determines the dependency-safe execution order and identifies invocations that may run concurrently. ReWOO separates its planner, workers, and solver, while LLMCompiler dispatches function calls when their required inputs become available [47, 125].

MacNet follows the topological order of its collaboration graph [89]. AgentScope provides distributed execution, parallelization, and fault-tolerance support for multi-agent applications, whereas Magentic-One monitors progress and replans after errors [22, 26]. Their stages commonly follow a predefined workflow or an initially constructed graph rather than being reorganized as runtime conditions change.

Plan selection and recovery. This decision concerns whether the system searches for an alternative design or repairs an ongoing execution. AFlow searches code-represented workflows, whereas AgentSquare searches combinations of planning, reasoning, tool-use, and memory modules [103, 141]. Their search spaces cover workflow or agent designs, not just runtime plans for one scheduled query. Reflexion uses linguistic feedback from previous trials, Self-Refine iteratively revises an output, and CRITIC validates and amends outputs through external tools [33, 63, 107]. Multi-agent debate compares responses over several interaction rounds [17]. These methods can support correction or verification within an execution process, but they do not necessarily modify all components of Π_E . Likewise, τ -bench evaluates whether agents interact with users, call tools, and follow policies reliably; it does not construct an execution plan [130]. For failed dataflow tasks, RDDs retain lineage so that a lost partition can be rebuilt from the transformations that produced it [136]. Agent invocations are harder to replay because a repeated model call may return a different result, while a repeated tool call may duplicate an external effect.

The five decisions interact closely: splitting work creates more invocations, specialized assignments may require additional communication, and verification introduces calls that can limit otherwise available parallelism. A denser graph supplies more intermediate results but also sends more messages and may spread incorrect ones. Several plan components may therefore need revision together without losing scheduled-unit coverage or logical dependencies. Table 3 maps representative methods to these decisions.

Table 3: Representative methods mapped to the five execution decisions.

Execution decision	Plan element	Representative methods	Main role	Typical limitation
Unit realization and coverage	λ_E	HuggingGPT [106]; ReWOO [125]; LLMCompiler [47]; AFlow [141]	Converts scheduled units into executable model, tool, or agent invocations.	Unit coverage, split-output aggregation, and replica consistency are rarely represented together.
Agent assignment	μ_E	CAMEL [55]; AutoGen [122]; MetaGPT [39]; ChatDev [88]; AgentVerse [9]; Magentic-One [22]	Delegates invocations according to agent roles, capabilities, or specialization.	Assignments are often predefined, task-specific, or weakly adapted to runtime conditions.
Invocation-graph construction	G_E	Dryad [42]; AutoGen [122]; MetaGPT [39]; ChatDev [88]; AgentVerse [9]; MacNet [89]; GPTSwarm [148]; AgentPrune [139]	Defines or optimizes dependency and communication links among invocations.	Some systems leave the distinction between required dependencies and optional communication implicit.
Runtime staging	σ_E	ReWOO [125]; LLMCompiler [47]; MacNet [89]; AgentScope [26]; Magentic-One [22]	Supports dependency-safe ordering, concurrency, distributed execution, and runtime coordination.	Stages commonly follow a predefined workflow or the initially constructed graph.
Plan selection and recovery	Π_E	<i>Design search:</i> AFlow [141]; AgentSquare [103]; <i>feedback or verification:</i> Reflexion [107], Self-Refine [63], CRITIC [33], multi-agent debate [17]; <i>dataflow recovery:</i> RDDs [136]	Searches alternative designs, corrects outputs, or reconstructs failed computation.	Lineage replay assumes that failed work can be repeated, while model and tool calls may return a different result or repeat an external effect.

Notes. A method may contribute to multiple execution decisions. The categories describe how methods relate to the proposed layer model rather than the terminology used in their original papers. Evaluation methods such as τ -bench [130] are discussed in the text but are not listed as plan-construction methods.

4.3 Challenges and Future Directions

Current systems often fix much of the physical execution plan before a task begins. Failures, unreliable outputs, or changing resources may require that plan to be revised while Planning’s logical units and dependencies remain fixed.

Runtime plan repair. ReWOO separates planning from tool observations, while LLMCompiler uses known dependencies to execute independent calls in parallel [47, 125]. Magentic-One replans after errors, and AgentScope provides fault-tolerant distributed execution [22, 26]. These systems handle repair within their own execution models. Repair therefore needs a record of completed invocations. Execution can then reassign failed work or revise later stages without losing unit coverage or repeating completed effects.

Communication graph construction. The communication graph controls which intermediate results reach each agent. GPTSwarm optimizes connectivity, MacNet studies DAG-based collaboration, and AgentPrune removes redundant messages from spatial-temporal graphs [89, 139, 148]. Revision preserves required dependencies and keeps an optional edge when its message is needed downstream. Other exchanges add latency and may spread errors.

Physical plan selection. AFlow and AgentSquare search agent workflows [103, 141]. Physical execution presents the narrower problem of selecting among realizations of the same logical plan. Completed runs can provide latency and failure estimates for later selections. Candidate plans still cover every scheduled unit and preserve its dependencies.

5 Action Layer

The Action Layer performs the final transformation in Eq. (1). It executes the invocations in Π_E through operations exposed in D , using M when a call depends on earlier task state. Action selects an executable operation, prepares and checks the call, and then performs it. The observed outcome is recorded in Z as a success, failure, or uncertain effect.

Running example. For v_1 , Action selects the X-ray write operation exposed through D . It grounds the patient and X-ray identifiers in the authorized task state and checks the call before execution. Action then records whether the source confirms the write in Z . When v_2 is dispatched, it uses the corresponding read operation to retrieve Sam’s stored X-ray.

5.1 Layer Model

The Action Layer can be understood in five steps:

$$\text{Resolve} \rightarrow \text{Ground} \rightarrow \text{Validate} \rightarrow \text{Invoke} \rightarrow \text{Verify}. \quad (3)$$

These steps recur across tool-learning methods, interface protocols, transaction mechanisms, and executable evaluations. An implementation may combine them; the decomposition lets us compare where a system selects a capability, checks a request, performs the call, and establishes its outcome.

Resolve and Ground. *Resolve* identifies a capability that can realize the dispatched operation. For data operations, the starting point is an operation handle in D ; for an external service, it may be an authorized API or tool entry. A valid binding refers to the intended object and exposes an operation

whose schema and effect match the plan. Protocols such as MCP standardize parts of this description, but schema exposure alone does not establish authorization or effect semantics [70]. *Ground* then constructs concrete arguments from the request and the task state available in M . Each consequential argument needs a traceable source in user input or observed state instead of relying only on model priors.

Validate and Invoke. *Validate* checks the grounded request before an external call is made. It first establishes that the request is well formed for the current resource and that the caller is authorized to produce the intended effect. Sensitive or conflicting operations may also require explicit approval or a resource-side guard. Write operations need a policy for duplicate delivery and uncertain outcomes when a true rollback is not available. *Invoke* transmits the validated request and records the response and any observable state change. For retryable state-changing operations, recording the intended effect before transmission helps distinguish a failed call from a call whose response was lost after the effect occurred.

Verify. *Verify* determines whether the intended outcome is supported by the available evidence. A returned success message is not, by itself, proof that a write took effect. Systems may instead use a subsequent read, a resource-side commit record, or another trusted attestation. Verification evidence retains its source and freshness, and an independent state observation is preferable when the invoked endpoint may be unreliable. The resulting outcome record distinguishes success, failure, partial effect, and an unknown outcome. It retains the evidence for that classification and reports any confirmed state change through Z . When observable channels share the same failure mode, or when evidence is incomplete or contradictory, retaining an unknown result is safer than silently retrying a potentially non-idempotent operation.

The five steps separate concerns often collapsed into “tool calling.” ACID properties remain those of the underlying resource. The grounded request and authority evidence remain linked to the invocation and its observed outcome.

5.2 Existing Work

Table 4 groups work by the part of the action path that it addresses. The marks distinguish mechanisms from benchmarks that exercise a step without enforcing it at runtime.

Capability resolution and grounding. Early tool-use research shows how language models can decide when to call a tool and construct its arguments. Toolformer learns tool-augmented generation from self-supervised examples, while Gorilla, ToolLLM, and TRICE extend this idea to larger and more varied tool spaces [85, 90, 91, 102]. ToolRerank and Re-Invoke focus more directly on retrieving or ranking a useful capability before a call is formed [10, 143]. These methods primarily address whether an appropriate capability and plausible arguments can be found; they generally

rely on the surrounding runtime for authorization and effect handling.

Request validation and protocols. ReAct interleaves reasoning, action, and observation, making tool feedback part of later decisions, while TOOLVERIFIER adds checks around generated tool interactions [66, 131]. MCP and A2A provide standard envelopes for advertising capabilities, exchanging structured requests, and coordinating tasks [1, 70]. Such interfaces reduce syntactic mismatch and improve interoperability, but an accepted request may still be semantically wrong, unauthorized, or unsafe for the current resource state. Protocol conformance and operation admissibility are therefore related but distinct concerns.

Executable invocation and verification. Database recovery provides concrete precedents for state-changing actions. ARIES supports database transaction recovery after failures, while Sagas divide long-running work into transactions and use compensating transactions when the full sequence cannot complete [27, 71]. Beldi records function invocations and operations in stateful serverless workflows, supporting exactly-once execution for operations managed by its runtime and transactions across functions [140]. Legacy external calls receive only at-least-once execution, a distinction that also matters when agents call services outside the data system. BridgeScope applies database privileges and transaction controls to LLM tool use [121]. In parallel, API-Bank, ToolTalk, ToolSandbox, GTA, StableToolBench, and Live API-Bench evaluate agents against executable tools, simulated state, or changing APIs [19, 21, 34, 56, 61, 116]. These benchmarks reveal whether an agent produced a valid call or achieved an observable outcome, but they do not enforce whether the call is admissible at runtime.

Safety and robustness. RoTBench studies changing tool interfaces, and AgentDojo focuses on prompt injection. ToolEmu and ToolSword examine unsafe or failed tool use [14, 95, 133, 134]. These benchmarks show that final task success can hide an inadmissible call or an incorrect external effect. Their evidence supports stage-specific evaluation, especially for validation and verification, while also showing that detection after execution is not a substitute for prevention before a call.

Existing work covers different parts of the action path. Tool-use methods and protocols form requests, while database mechanisms and benchmarks examine their effects. Few systems carry a capability binding through authorization and invocation to evidence about the result. The five-step model exposes these handoffs.

5.3 Challenges and Future Directions

At invocation time, Action binds an operation and checks the caller’s authority. It may later need to recover from an uncertain effect while treating returned content as untrusted.

Table 4: Representative action mechanisms and evaluation evidence across the five action steps.

Research focus	Primary object	Representative works	Evidence role	Res.	Grd.	Val.	Inv.	Ver.
Capability resolution and grounding	Learned tool use	Toolformer, Gorilla, ToolLLM, TRICE [85, 90, 91, 102]	Method	✓	✓	–	✓	◦
	Catalog retrieval and ranking	ToolRerank, Re-Invoke [10, 143]	Method	✓	◦	–	–	–
Request validation and protocols	Interaction and request checking	ReAct, ToolVerifier [66, 131]	Method	✓	✓	◦	✓	◦
	Capability and task envelopes	MCP, A2A [1, 70]	Specification	✓	–	◦	✓	◦
Executable invocation and verification	Transaction and effect handling	ARIES, Sagas, Beldi, BridgeScope [27, 71, 121, 140]	Model/system	–	–	✓	✓	✓
	Executable state and outcomes	API-Bank, ToolTalk, ToolSandbox, GTA, StableToolBench, Live API-Bench [19, 21, 34, 56, 61, 116]	Benchmark	◊	◊	–	◊	◊
Safety and robustness	Failure handling and least privilege	RoTBench, ToolEmu, AgentDojo, ToolSword [14, 95, 133, 134]	Benchmark	◊	◊	◊	◊	◊

Notes. Res. = resolution; Grd. = grounding; Val. = pre-invocation schema, constraint, authorization, or transaction-policy checking; Inv. = external or simulated interaction; Ver. = outcome, state, safety, or postcondition verification. ✓ denotes direct implementation or standardization by at least one cited artifact; ◦ denotes partial or indirect support; ◊ denotes direct exercise or measurement rather than enforcement; – denotes no direct evidence coded here, not demonstrated inability. Marks aggregate within a row and do not imply that one artifact spans every marked stage.

Action binding. A syntactically valid call may still invoke the wrong tool or address the wrong object. ToolRerank and Re-Invoke retrieve capabilities from large collections. ToolVerifier checks tool selection and arguments, while RoTBench tests robustness to noisy tool information [10, 66, 134, 143]. Action therefore binds the planned operation to a descriptor in D and checks that its arguments identify the intended object and meet the operation’s preconditions. An ambiguous binding goes to the user for confirmation before invocation.

Recovery from uncertain outcomes. A failed response does not show whether the operation took effect. A time-out may follow an external write, so retrying can duplicate it. ToolTalk and ToolSandbox expose these state dependencies in multi-step tool use [21, 61]. An external service may provide neither a commit record nor a rollback operation. Action therefore inspects the resulting state before retrying. An idempotent call can then be repeated, while other calls may need compensation or an outcome recorded as unknown. A repair that changes assignments or dependencies returns to Execution.

Authorization and untrusted outputs. An authorized call can still return malicious instructions. AgentDojo studies indirect prompt injection, while ToolEmu and ToolSword evaluate unsafe tool use [14, 95, 133]. Before invocation, Action checks the operation against the user’s permissions and current task. Work beyond that scope requires explicit approval. Afterward, Action retains the returned content as tool-attributed evidence rather than an instruction for the next step. Evaluation examines executed calls and state changes, not only the final answer. ToolSandbox evaluates state mile-

stones, while StableToolBench provides stable executable services for reproducible tests [34, 61].

6 Memory Layer

The Memory Layer supplies M in the shared state of Eq. (2). It decides which task records are retained and which records a layer may read. The Data Interface persists and indexes these records, while Memory constructs the authorized view presented as M . This section examines how memory supports task history and coordination across agents.

Running example. As Patient A’s request proceeds, Memory links the interpreted request, scheduled unit u_1 , invocation v_1 , and the outcome of the X-ray write. Before the dependent read v_2 , M can provide the authorized write record and its DataObject reference. The X-ray itself remains accessible through D .

6.1 Layer Model

Let $\mathcal{H}_t$ denote the task records retained at time t . For a particular request, Memory exposes

$$M_t = \text{Access}_M(\mathcal{H}_t, c_t), \quad (4)$$

where c_t describes the request and its access conditions. Thus, M_t is an authorized view of the relevant retained records.

Each record has a stable identifier and is tied to the task step that produced it. Application data remains in D ; memory records refer to the corresponding DataObjects so that workflow state can be traced to its source. A later record may derive from or supersede an earlier one, preserving the

history needed for inspection and recovery. Memory selects the records exposed in each view and governs their retention. The Data Interface persists and indexes these records and their references.

6.2 Existing Work

We retain only method families that directly inform database state management or multi-agent coordination. Table 5 summarizes which ACDS layers can reuse each mechanism and what remains unsupported for database workloads.

Episodic and feedback memory. Episodic memory stores task-specific trajectories rather than general facts. It lets Planning retrieve comparable workloads and lets later layers consult earlier outcomes. MemGPT supports long-running interaction by moving prior records between the limited model context and external storage [80]. Generative Agents stores observations in a memory stream and retrieves them using recency, relevance, and importance, providing a basic mechanism for selecting task history [83]. Reflexion stores linguistic feedback from previous trials and reuses it to revise later decisions, making failure experience available without parameter updates [107]. G-Memory organizes multi-agent histories into interaction, query, and insight graphs, allowing both detailed collaboration traces and reusable knowledge to be retrieved [138]. In ACDS, an episode also remains linked to the task path that produced it, from its query unit through the resulting action. This link lets a later task inspect how the episode was produced before reusing it.

Workflow and artifact memory. Workflow memory stores reusable procedures and the artifacts they produce. In ACDS, a retrieved workflow can guide Planning and carry intermediate results across execution stages. Agent Workflow Memory reuses workflows extracted from prior solutions [120], and MetaGPT and ChatDev retain role-specific artifacts across staged software workflows [39, 88]. To remain reusable, a workflow record needs to name its DataObjects and detect later changes.

Shared coordination memory. Shared coordination memory records task ownership and the state that agents need to exchange. Execution consults it when assigning or synchronizing invocations, Planning uses it to see whether work has completed or become blocked, and Action publishes outcome records. LLM-based blackboard systems let agents post partial results and hypotheses asynchronously [97]. Linda tuple spaces use associative read, write, and take operations that decouple producers and consumers and can support atomic task claims [28]. Event logs preserve ordered task transitions for replay and asynchronous consumption [23, 49]. ACDS also needs rules for concurrent updates.

Governed memory and provenance. Governed memory controls who may read or update retained records and when

those records are refreshed or removed. Task records can contain private data or stale claims, so the same controls apply across ACDS. Collaborative Memory separates private and shared memories and introduces dynamic access control for cross-user or cross-agent sharing [93]. Event sourcing provides an append-only transition history, while provenance models connect derived records to source data and transformations [23, 69]. RBAC and ABAC provide policy models for role- and attribute-aware views [41, 99]. These mechanisms do not decide which records the agent retains or when a derived record has become stale.

The methods above retrieve prior experience but leave concurrent updates and source changes largely to the surrounding system. Database mechanisms can serialize updates and track source changes, but they do not decide which earlier experiences an agent retains or retrieves.

6.3 Challenges and Future Directions

Reusing a record requires knowing where it came from and whether the data or outcome behind it has changed. Memory makes that decision while several agents update the history and each request retrieves only part of it.

Record lineage and invalidation. A summary or reused result may depend on several task steps and DataObjects. If one of those sources changes, the text can still appear relevant even though the result is no longer valid. A single timestamp cannot represent these dependencies. A reusable memory record therefore needs a stable identifier, derivation links to its sources, and a distinction between observed evidence and model inference. PROV represents derivations, while event sourcing retains state transitions [23, 69]. When the Data Interface observes a source update, or Action records a revised outcome, Memory can invalidate or recheck the affected records.

Concurrent updates. Several agents may claim the same unit or update the same artifact. They may also publish incompatible results. These records need different update rules. An atomic claim prevents two agents from owning the same unit. Progress can be appended to an ordered log, while incompatible results remain separate until they are resolved. Linda tuple spaces support exclusive task claims, and event logs preserve ordered progress [28, 49]. Memory defines the rule for each record type, and the Data Interface enforces it when the record is stored.

Memory View selection. Task history outgrows the model context. Similarity search may return stale records or omit context needed to use them. Memory first applies c_i 's access conditions and removes records with changed sources. It can then retrieve original records or materialized summaries, with every summary linked to its evidence. The Data Interface supplies the index lookups, and Inference Infrastructure

Table 5: Representative memory methods relevant to ACDS. A checkmark indicates a mechanism that the corresponding layer could reuse after integration.

Memory Category	Method	Planning	Execution	Action	Data Intf.	Current Limitations for the ACDS System
Episodic and Feedback Memory	MemGPT [80]	✓				Its tiered context does not link retained records to task steps or source data.
	Generative Agents [83]	✓				Free-form memories are not tied to task records or their source data.
	Reflexion [107]	✓	✓	✓		Trial-level feedback may be reused after its supporting evidence or database state has changed.
	G-Memory [138]	✓	✓	✓		Hierarchical traces do not define how concurrent agents update shared task state.
Workflow and Artifact Memory	Agent Workflow Memory [120]	✓	✓	✓		A stored workflow may be reused after its source data or permissions have changed.
	MetaGPT / ChatDev [39, 88]	✓	✓			Their artifacts do not share common access or update rules.
Shared Coordination Memory	MAS Blackboard [97]	✓	✓			The workspace does not prevent duplicate task claims or restrict which records each agent can read.
	Linda [28]		✓	✓		Tuple matching does not directly retrieve plans or evidence by their content.
Governed Memory and Provenance	Collaborative Memory / PROV [69, 93]	✓	✓	✓	✓	Access control and provenance are handled separately, leaving memory refresh and concurrent updates unresolved.

materializes the selected M_i . Evaluation can test whether M_i contains the evidence needed for the task while excluding stale or unauthorized records, then report the resulting context size and retrieval latency.

7 Data Interface Layer

The Data Interface Layer supplies D in the shared state of Eq. (2). It represents external resources and ACDS-managed records as source-backed DataObjects and exposes the operations supported by their source systems. This section first describes how D is constructed. It then reviews the databases, stores, indexes, and services behind the view. Figure 4 summarizes this process.

Running example. For Patient A (Sam), D links the patient record to the relevant X-ray and report objects and exposes the permitted write and read operations. When Action uses one of these operations, the Data Interface preserves the source reference for the result. A confirmed change refreshes the affected object in D .

7.1 Layer Model

Within D , external resources $\mathcal{R}$ and ACDS-managed records or artifacts are presented through their logical interfaces, while source systems retain their physical layouts and behavior. The view contains DataObjects, typed relationships, and operation handles. A DataObject has a stable identity and source reference and may carry source metadata and usage conditions. An operation handle describes an operation on an object.

Source systems own authoritative application data, while ACDS-managed stores persist task records and intermediate artifacts. Derived indexes may become stale when their sources change. The Data Interface records stable handles and source links in D . Memory uses these handles to construct M , and Action invokes operations through D to produce Z . Read results may refresh metadata and evidence links, while verified changes update DataObjects.

7.2 Existing Work

Source systems and native semantics. PostgreSQL and F1 illustrate transactional systems, while Druid and DuckDB provide analytical interfaces [86, 92, 108, 128]. Bigtable and MongoDB expose key-value and document data, RDF-3X and gStore provide graph access, and SciDB works over arrays [7, 72, 75, 111, 149]. File and lake systems preserve source formats. Google File System illustrates scalable file storage, data lake systems organize heterogeneous repositories, and Delta Lake adds transactional table snapshots over object storage [3, 29, 36].

Derived indexes and service-backed resources. BM25 and DPR provide lexical and dense retrieval, while FAISS and Milvus manage vector indexes that need to remain traceable to their source data [43, 44, 81, 94, 117]. Federation systems and dataspace provide access across independently managed sources [24, 25, 37, 48]. APIs and services expose state and operations governed by their source systems, while ACDS-managed stores persist retained records and intermediate artifacts that the Data Interface represents as DataObjects in D , together with their source references and stable handles.

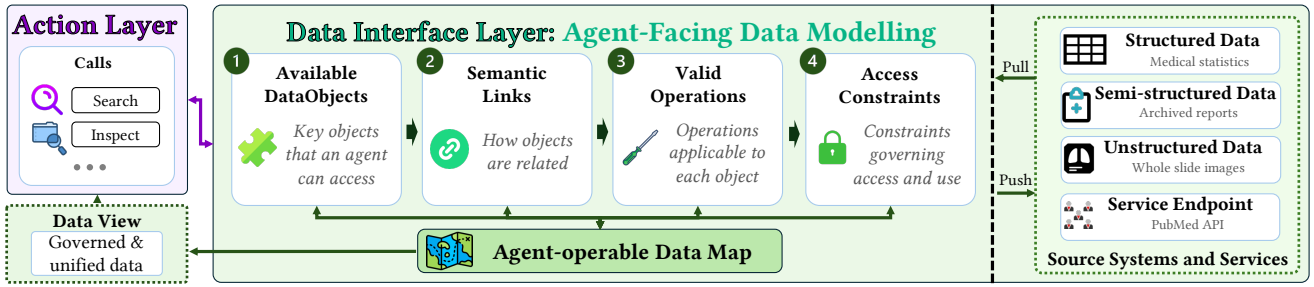


Fig. 4: Construction and use of the Data View D . The Data Interface maps source systems and ACDS-managed stores to DataObjects, records relationships between these objects, and exposes supported operations. Memory uses the resulting handles when constructing M , while Action invokes the operations through D . Confirmed outcomes refresh the affected objects and their source links

These systems supply the data and operations exposed by the Data Interface. The following discussion examines how agents find, operate on, and trace resources. Table 6 compares these capabilities.

Retriever-mediated RAG. Conventional RAG, BM25, and DPR expose data mainly through retriever-mediated interfaces [44, 54, 94], including the pipeline query $\rightarrow$ retriever $\rightarrow$ top- k chunks. This interface is effective for many question-answering tasks but is coarse for agentic data exploration. A small k may omit necessary evidence, while a large k adds redundant or conflicting context. The returned chunks also do not reveal which operations the source supports or preserve enough information to verify a result against that source.

Direct corpus interaction. Direct Corpus Interaction (DCI) lets agents inspect and process files through executable commands [57], while GrepSeek trains search agents to use a shell interface over a corpus [98]. These works go beyond fixed top- k retrieval but expose mainly file-system operations.

Dataspaces, data catalogs, and data lake management. Dataspace systems propose pay-as-you-go integration over heterogeneous sources without requiring full upfront semantic integration [24, 37]. Data lake management systems use catalogs to support discovery and provenance in heterogeneous repositories [36, 74]. These interfaces target analysts and applications, with little support for agent actions or result tracing.

Enterprise semantic indexes. The Microsoft 365 Copilot Semantic Index uses semantic and permission-aware access to ground responses in organizational data [68]. Its interface is oriented toward context retrieval rather than object-level operations and source-level traceability.

Data discovery systems. Data discovery work studies how to find relevant, joinable, or unionable datasets from large table collections. LakeBench benchmarks joinable and unionable table discovery [110]. Starmie learns contextualized column representations for table union search [20], while SANTOS

uses semantic relationships between columns [45]. These methods support object discovery and navigation but remain mainly table-centric, with limited support for operations and provenance across heterogeneous resources.

Graph-based interfaces. GraphRAG constructs graph-based indexes and community summaries over document collections to support query-focused summarization [18], while Docs2KG constructs knowledge graphs from heterogeneous documents [112]. These systems support multi-hop navigation and cross-document organization. However, graph structure alone does not specify which operations are valid or how their results remain tied to source evidence.

Semantic operator systems. Recent systems expose LLM-based relational operations and compare alternative ways to execute them [58, 84, 87, 96, 104]. They provide useful execution primitives once a semantic operation has been selected, but generally assume that the target DataObject and operation are already known.

The interface families cover different parts of D . Retrievers and catalogs help find data, graph systems add relationships, and semantic operators process selected objects. They differ in whether results remain linked to source data and whether those links are refreshed.

7.3 Challenges and Future Directions

D needs operation descriptions for its objects and links that remain valid as source systems change.

DataObject descriptions. A name and schema do not show whether an object supports the planned operation. For example, a table and an API object may expose similar fields but allow different reads and updates. A descriptor in D therefore combines the source schema with operation handles and usage conditions. Model-generated annotations can add task-specific terms, provided they are marked as inferred and checked against the source.

Table 6: Comparison of existing data-access interfaces for agents.

Category	Representative works	Main interface	Obj.	Links	Ops.	Prov.	Constr.	Main limitation
Retriever-mediated RAG	RAG [54], BM25 [94], DPR [44], and semantic retrieval systems	Top- k chunks or passages	✓					Does not expose source operations or reliable evidence links.
Direct corpus interaction	DCI [57], GrepSeek [98]	File and shell operations	✓		✓			Mainly exposes raw corpus or file-system operations; costly to explore and difficult to govern across heterogeneous resources.
Dataspaces, catalogs, and data lake management	Dataspaces [24, 37]; data lake management [36, 74]	Metadata catalogs and pay-as-you-go integration	✓	✓		✓	✓	Mostly designed for human analysts or application-level data management; does not directly expose object-level interfaces for agents.
Enterprise semantic indexes	Microsoft 365 Copilot Semantic Index [68]	Semantic indexes over organizational data	✓	✓			✓	Primarily optimized for retrieving relevant context rather than directly exposing valid operations and source-level traceability.
Data discovery systems	Starmie [20], SANTOS [45], LakeBench [110]	Relevant, joinable, or unionable table discovery	✓	✓				Focused on tabular discovery rather than governed operations over heterogeneous resources.
Graph-based interfaces	GraphRAG [18], Docs2KG [112]	Graph indexes and summaries	✓	✓				Graph structure alone does not define source operations or maintain evidence links.
Semantic operator systems	LOTUS [84], Palimpzest [58], DocETL [104], Abacus [96], Sema [87]	LLM-based relational operators			✓		✓	Usually assumes that the target DataObject and operation have already been selected; weaker at upstream discovery and evidence traceability.

Notes. Obj. = object discovery; Links = typed semantic links; Ops. = exposed operation handles or valid object-level operations; Prov. = provenance or evidence traceability; Constr. = source-declared usage constraints. ✓ indicates direct support. Blank cells indicate that the surveyed representative work does not provide direct evidence of the corresponding capability.

Cross-source object resolution. Different systems may store records for the same entity without sharing a key. Similarity alone can merge the wrong objects, which becomes dangerous when Action later invokes an operation handle. In the running example, the records for Patient A in the EHR and imaging store cannot be linked by name alone. Starmie and SANTOS discover related tables, while Docs2KG constructs links from heterogeneous documents [20, 45, 112]. These methods can propose candidate relations, but they do not establish object identity on their own. For each accepted link, D retains the source references and the evidence used to establish the match. A derived index likewise points back to the objects from which it was built.

Data View maintenance. An object handle or evidence link can become stale during a long-running workflow. Sources update independently. Some publish change events, while others reveal a change only when queried. The Data Interface can consume these events when available and recheck a handle before it is used. An outcome record in Z refreshes the affected DataObjects and evidence links only when it confirms the corresponding read or state change. The resulting changes are then passed to Memory. Evaluation can count

calls attempted through stale handles and results that cannot be traced back to their source.

8 Inference Infrastructure Layer

The Inference Infrastructure Layer supplies I in the shared state of Eq. (2). It places model calls, data operations, and managed state on compute and storage resources. It also schedules the required communication. These physical decisions support all ACDS layers while preserving the agent assignments and dependencies in Π_E . This section examines compute scheduling, distributed storage, and data transfer during inference.

Running example. The infrastructure can place v_1 and the independent v_3 on separate workers. After v_1 stores Sam’s X-ray, it makes the state required by v_2 available to the worker that will perform the read. These choices determine physical placement and transfer without changing the dependency $v_1 \rightarrow v_2$ or the assigned agents.

8.1 Layer Model

At runtime step t , the Runtime Realization is represented as

$$I_t = \langle I_t^{\text{cmp}}, I_t^{\text{sto}}, I_t^{\text{net}} \rangle.$$

The three components describe compute, storage, and communication decisions. **The compute component** I_t^{cmp} maps model calls and data operations to workers and devices. It decides when a ready operation starts and may batch compatible model calls. The resulting order follows the dependencies supplied by the upper layers. **The storage component** I_t^{sto} places ACDS-managed records and indexes, as well as the runtime state used by model calls. It chooses a storage node and tier for each item and may move a copy closer to its next consumer. Logical ownership and access conditions remain attached to the item. **The communication component** I_t^{net} moves required inputs and runtime state between the selected locations. Upper-layer dependencies determine what has to be transferred, while I_t^{net} chooses the timing and performs the synchronization required by distributed inference. Runtime feedback, such as delays or failures, may trigger new compute, storage, or transfer decisions.

8.2 Existing Work

We organize existing work by the compute, storage, and communication components of I_t . Some systems optimize the physical runtime directly; agent frameworks instead expose dependencies and state that an infrastructure scheduler can use. The work reviewed here concentrates on model execution and transient model state, with less support for data-processing operations and ACDS-managed storage. Table 7 groups representative systems by their primary component. **Compute scheduling for agent workloads.** The compute component I_t^{cmp} places and schedules model calls and data-processing operations over heterogeneous resources. Agentix treats agent programs as first-class scheduling units and uses program-level execution progress to prioritize and preempt their model calls. AI Metropolis dynamically tracks dependencies among agents and enables out-of-order execution to improve parallelism in multi-agent simulations [62, 124]. General LLM serving supplies lower-level scheduling mechanisms. Orca schedules generation at the iteration level and selectively batches requests, while DistServe places prefill and decoding on separate resources and chooses parallelism for each phase [135, 144]. LLMCompiler constructs execution plans for tool-using workloads and dispatches independent function calls in parallel [47]. Application frameworks such as AutoGen, MetaGPT, and OpenHands expose workflow stages and tool boundaries that a physical scheduler can use [39, 119, 122]. These systems schedule model inference or expose workflow structure, but do not place model calls and ACDS data operations under one physical scheduler.

Distributed storage management. The storage component I_t^{sto} decides where runtime state resides and whether to move or recompute it. vLLM uses PagedAttention to manage KV-cache memory and share cached blocks within and across requests [52]. Prompt Cache precomputes and reuses attention states for modular prompt segments, while BumbleBee selectively summarizes streaming KV-cache states to support continuously growing contexts [31, 51]. For agent-specific workloads, KVFlow derives cache-management decisions from agent workflow structure, using execution-aware eviction and proactive prefetching to preserve KV states likely to be reused [82]. Agentic Plan Caching caches and adapts plans from completed agent executions [142]. These systems reuse model state or workflow artifacts but do not manage other ACDS runtime objects.

Communication and shared-context coordination. The communication component I_t^{net} manages transfers and synchronization among runtime components. At the application level, AutoGen and OpenHands expose message and event dependencies that the physical runtime has to realize [119, 122]. At the physical-runtime level, KVCOMM reuses overlapping contexts by aligning KV states across related agents, while Q-KVComm transmits adaptively compressed KV-cache representations between agents [50, 132]. ALTO complements these approaches by streaming partial model outputs and coordinating routing across stages of compound AI pipelines [100]. These mechanisms cover KV reuse, compressed transfer, and pipeline streaming. General worker-storage transfers for ACDS data operations receive less attention.

8.3 Challenges and Future Directions

Infrastructure receives operations and dependencies from the upper layers. The remaining decisions concern when those operations run and where their required state is kept and transferred.

Joint runtime scheduling. Model calls and data operations can lie on the same dependency path but enter different runtime queues. An LLM server sees its model requests, while a data engine sees its own queries. Neither alone knows when a delayed data operation blocks a downstream model call. A shared runtime interface could expose ready work and completion events to Infrastructure, which then decides what starts next while each engine retains its local execution mechanisms. For work in Π_E , this scheduling changes physical workers and start times without changing the assigned agents or logical dependencies [47, 62, 124, 135, 144].

Runtime-state placement. Fast storage cannot hold every state that a later operation may reuse. Keeping a KV cache near a device can avoid repeated prefill, whereas evicting it too early repeats work. A selected Memory View or index may remain useful for longer and reside in distributed storage. Placement can use known downstream consumers and

Table 7: Representative infrastructure work organized by its primary physical-resource concern.

Category	Representative systems	Main role	Current gap
Compute scheduling	Agentix [62]; AI Metropolis [124]; Orca [135]; DistServe [144]; LLMCompiler [47]	Schedules model calls and exposes workflow dependencies for parallel execution.	Model serving and general data operations are not scheduled together.
Storage management	vLLM [52]; Prompt Cache [31]; BumbleBee [51]; KVFlow [82]; Agentic Plan Caching [142]	Manages the placement and reuse of runtime state.	Each method manages a particular type of cached state rather than the full ACDS runtime state.
Communication	AutoGen [122]; OpenHands [119]; KV-COMM [132]; Q-KVComm [50]; ALTO [100]	Coordinates transfers between runtime stages.	Data movement between model execution and data access remains weakly coordinated.

observed workflow progress to decide where to keep an item. State needed by the next stage remains near its worker, while other state can move to a lower storage tier or be rebuilt later. The same physical policy places ACDS-managed records and indexes, while the Data Interface retains their logical identity and access conditions [31, 51, 52, 82].

Data movement. After workers and storage locations are chosen, the required state has to reach each consumer before it can run. A large KV transfer may delay a worker longer than rebuilding the state at its destination. Scheduling transfers separately from computation can also leave devices idle. Execution supplies the dependency and communication edges in G_E . Infrastructure can use those edges to prefetch state and overlap transfer with current computation. Measurements of transfer and reconstruction time determine whether moving or rebuilding a state is faster. Distributed model calls use the same schedule for their required synchronization [50, 100, 132].

9 Cross-Layer Challenges and Research Directions

The layer-specific discussions point to four questions that span more than one layer. Table 8 locates these questions in ACDS. The discussion below considers the common foundations that are still missing.

Portable task records and provenance. An agent system can usually interpret the records produced by its own runtime. A task becomes harder to resume in another framework after those records have been summarized, merged, or rewritten. Generic provenance describes how one artifact derives from another, but an agent task also depends on the evidence used to interpret a request and the authority under which an operation was attempted. That authority may need to be checked again when a record is reused. PROV provides a common model for provenance interchange, while MCP provides a common interface for exposing resources and tools [40, 69]. Neither yet describes a task record that remains meaningful throughout an agent workflow.

An open and extensible interchange model would give agent task records a portable form. A small common core could preserve source references and derivation relations, while prompts and domain-specific fields remain local. Compact provenance and selective disclosure would limit both

trace size and exposure of private data. Reference adapters and conformance tests could then check whether a task can move between independently developed systems without losing the evidence behind an answer or the authority behind an action. ACDS offers a vocabulary for these tests by identifying the records exchanged along a task.

Adaptive optimization for agent workloads. Database optimizers rely on operators with defined semantics and rewrites that preserve results. Agent workloads weaken both assumptions. Repeating a model call may change its answer, and combining requests can change the context seen by the model. An external write also cannot be shared like a read. Execution cost depends on the same choices. For example, one shared call may save work but create a longer context and delay every request that depends on it. Eddies adapts operator routing from runtime observations, and Bao learns from completed queries to steer an existing optimizer [4, 64]. Palimpzest and DocETL extend plan search and rewriting to AI data processing [58, 104], but they cover only part of a changing agent workload.

Research on agent workload optimization needs a common problem formulation. Such a formulation would state which transformations preserve an acceptable task result when requests have different access conditions or external effects. Empirical models could then estimate outcome quality and execution cost while reporting uncertainty. Public optimization traces could record alternative plans and their measured outcomes, allowing cost models and adaptive policies to be compared without repeating every live service call. These traces would support methods that decide when runtime evidence justifies replanning and when the expected gain is smaller than the replanning overhead. ACDS separates logical alternatives from the physical measurements used to compare them.

Transactional semantics for agent workflows. An agent task may extend beyond the transaction boundary of every individual system it uses. A timeout after an external write leaves the caller unable to tell whether retrying will complete the task or repeat the effect. Sagas address long-running transactions through compensating transactions, while Beldi supplies transactional support when its runtime controls the functions and their state [27, 140]. Agent workflows complicate both settings because the next operation may be selected only after model reasoning, and an independently managed

Table 8: Cross-layer research directions and the ACDS layers they involve. ✓ denotes a direct role, ◦ a supporting role, and a blank limited involvement.

Research direction	Perception	Planning	Execution	Action	Memory	Data Interface	Inference Infra.
Portable task records and provenance	✓	✓	◦	✓	✓	✓	◦
Adaptive optimization for agent workloads	◦	✓	✓	◦	◦	◦	✓
Transactional semantics for agent workflows	◦	◦	◦	✓	✓	✓	◦
Layer-aware benchmark construction [11, 46, 65]	✓	✓	✓	✓	✓	✓	✓

service may expose neither a transaction nor a reliable way to query the outcome.

One research direction is a task-level correctness model for open agent workflows. Such a model would define partial completion and unresolved effects, including when retry, compensation, or human resolution is admissible. Effect and recovery contracts form a second line of work. Published with an operation description, a contract could state how completion is observed and which remedies the service supports. Fault-injection environments would then let researchers compare recovery methods against the same starting state. ToolTalk and ToolSandbox provide early settings for stateful tool interaction [21, 61]. Broader testbeds can add independently managed services and failures that occur after an external effect. Together, these mechanisms would let an agent continue under the guarantees that each service actually provides.

Layer-aware benchmark construction. Many agent benchmarks score only the final response or terminal environment state. That score does not reveal where the first error occurred, even though omitted evidence, an invalid plan, and an incorrect external effect require different remedies. The intermediate records needed for diagnosis are expensive to collect because realistic tasks depend on changing source state and observable side effects. Synthetic tasks can increase scale, but they may reproduce the assumptions of the models that generated them. Stateful executable benchmarks such as ToolSandbox improve realism [61], yet current benchmarks often omit the evidence needed to study the handoffs between data-system components.

Benchmark construction can combine model-assisted generation with human validation. Dynabench uses models in the data-collection loop to help annotators find difficult cases, while Chatbot Arena shows how live human feedback can broaden evaluation at scale [11, 46]. DataPerf adds a community-maintained platform in which new challenges can be contributed over time [65]. ACDS can guide the annotation. Models can propose candidate tasks and variations of observed failures, after which users or domain experts

verify whether a task is realistic and whether its expected effect is correct. Each instance can retain the records exchanged at ACDS handoffs and mark the first point at which a run becomes invalid. Later collection rounds can expand the range of data sources and task durations, then introduce permission changes or partial failures. This process would enlarge the benchmark while preserving labels that support both end-to-end scoring and failure diagnosis.

10 Conclusion

This survey examined how data systems can support agent workflows that evolve across model calls, tools, and external resources. ACDS organizes the relevant work into four task-processing layers and three supporting layers, with explicit objects exchanged between them. Existing systems provide mechanisms within individual layers, but often leave the handoffs among plans, task state, external effects, and physical resources implicit or tied to one runtime. ACDS identifies these handoffs as data-system problems in long-running agent tasks with traceable state and controlled effects.

References

1. Agent2Agent Protocol: Agent2Agent (A2A) protocol specification, version 0.2.6 (2025). URL <https://a2a-protocol.org/v0.2.6/specification/>
2. Ali, M.A., Dornaika, F., Charafeddine, J.: Agentic AI: a comprehensive survey of architectures, applications, and future directions. *Artif. Intell. Rev.* **59**(1), 11 (2026)
3. Armbrust, M., Das, T., Paranjpye, S., Xin, R., Zhu, S., Ghodsi, A., Yavuz, B., Murthy, M., Torres, J., Sun, L., Boncz, P., Mokhtar, M., Hovell, H.V., Ionescu, A., Luszczak, A., Switkowski, M., Ueshin, T., Li, X., Szafranski, M., Senster, P., Zaharia, M.: Delta lake: High-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* **13**(12), 3411–3424 (2020)

4. Avnur, R., Hellerstein, J.M.: Eddies: Continuously adaptive query processing. In: SIGMOD Conference, pp. 261–272. ACM (2000)
5. Biswal, A., Jha, S., Guestrin, C., Zaharia, M., Gonzalez, J.E., Kamsetty, A., Liu, S., Patel, L.: Text2sql is not enough: Unifying AI and databases with TAG. In: CIDR. www.cidrdb.org (2025)
6. Buneman, P., Khanna, S., Tan, W.C.: Why and where: A characterization of data provenance. In: ICDT, *Lecture Notes in Computer Science*, vol. 1973, pp. 316–330. Springer (2001)
7. Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A., Gruber, R.E.: Bigtable: A distributed storage system for structured data. *ACM Trans. Comput. Syst.* **26**(2), 4:1–4:26 (2008)
8. Chen, J., DeWitt, D.J., Tian, F., Wang, Y.: NiagaraCQ: A scalable continuous query system for internet databases. In: SIGMOD Conference, pp. 379–390. ACM (2000)
9. Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C., Yu, H., Lu, Y., Hung, Y., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., Zhou, J.: Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In: ICLR. OpenReview.net (2024)
10. Chen, Y., Yoon, J., Sachan, D.S., Wang, Q., Cohen-Addad, V., Bateni, M., Lee, C., Pfister, T.: Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In: EMNLP (Findings), *Findings of ACL*, vol. EMNLP 2024, pp. 4705–4726. Association for Computational Linguistics (2024)
11. Chiang, W., Zheng, L., Sheng, Y., Angelopoulos, A.N., Li, T., Li, D., Zhu, B., Zhang, H., Jordan, M.I., Gonzalez, J.E., Stoica, I.: Chatbot arena: An open platform for evaluating llms by human preference. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 8359–8388. PMLR / OpenReview.net (2024)
12. Dai, H., Wang, B., Wan, X., Dai, B., Yang, S., Nova, A., Yin, P., Phothilimthana, P.M., Sutton, C., Schuurmans, D.: UQE: A query engine for unstructured databases. In: NeurIPS (2024)
13. Databricks: Genie Spaces (2026). URL <https://docs.databricks.com/aws/en/genie>
14. DeBenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: NeurIPS (2024)
15. Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., Su, Y.: Mind2web: Towards a generalist agent for the web. In: NeurIPS (2023)
16. Deng, Z., Guo, Y., Han, C., Ma, W., Xiong, J., Wen, S., Xiang, Y.: AI agents under threat: A survey of key security challenges and future pathways. *ACM Comput. Surv.* **57**(7), 182:1–182:36 (2025)
17. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 11,733–11,763. PMLR / OpenReview.net (2024)
18. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. *CoRR* **abs/2404.16130** (2024)
19. Elder, B., Murthi, A., Kang, J., Naik, A., Basu, K., Kate, K., Contractor, D.: Live api-bench: 2500+ live apis for testing multi-step tool calling. In: EACL (Volume 1: Long Papers), pp. 3092–3124. Association for Computational Linguistics (2026)
20. Fan, G., Wang, J., Li, Y., Zhang, D., Miller, R.J.: Semantics-aware dataset discovery from data lakes with contextualized column-based representation learning. *Proc. VLDB Endow.* **16**(7), 1726–1739 (2023)
21. Farn, N., Shin, R.: Tooltalk: Evaluating tool-usage in a conversational setting. *CoRR* **abs/2311.10775** (2023)
22. Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Zhu, E., Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., Alber, J., Chang, P., Loynd, R., West, R., Dibia, V., Awadallah, A., Kamar, E., Hosn, R., Amershi, S.: Magentic-one: A generalist multi-agent system for solving complex tasks. *CoRR* **abs/2411.04468** (2024)
23. Fowler, M.: Event sourcing (2005). URL <https://martinfowler.com/eaDev/EventSourcing.html>
24. Franklin, M.J., Halevy, A.Y., Maier, D.: From databases to dataspace: a new abstraction for information management. *SIGMOD Rec.* **34**(4), 27–33 (2005)
25. Gadepally, V., Chen, P., Duggan, J., Elmore, A.J., Haynes, B., Kepner, J., Madden, S., Mattson, T., Stonebraker, M.: The bigdawg polystore system and architecture. In: HPEC, pp. 1–6. IEEE (2016)
26. Gao, D., Li, Z., Kuang, W., Pan, X., Chen, D., Ma, Z., Qian, B., Yao, L., Zhu, L., Cheng, C., Shi, H., Li, Y., Ding, B., Zhou, J.: Agentscope: A flexible yet robust multi-agent platform. *CoRR* **abs/2402.14034** (2024)
27. Garcia-Molina, H., Salem, K.: Sagas. In: SIGMOD Conference, pp. 249–259. ACM Press (1987)
28. Gelernter, D.: Generative communication in linda. *ACM Trans. Program. Lang. Syst.* **7**(1), 80–112 (1985)
29. Ghemawat, S., Gobioff, H., Leung, S.: The google file system. In: SOSP, pp. 29–43. ACM (2003)
30. Giannikis, G., Alonso, G., Kossmann, D.: Shareddb: Killing one thousand queries with one stone. *Proc. VLDB Endow.* **5**(6), 526–537 (2012)
31. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt cache: Modular attention reuse for low-latency inference. In: MLSys. mlsys.org (2024)
32. GitHub: GitHub Copilot (2026). URL <https://github.com/features/copilot>

33. Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., Chen, W.: CRITIC: large language models can self-correct with tool-interactive critiquing. In: ICLR. OpenReview.net (2024)
34. Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., Liu, Y.: Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In: ACL (Findings), *Findings of ACL*, vol. ACL 2024, pp. 11,143–11,156. Association for Computational Linguistics (2024)
35. Gupta, A., Mumick, I.S., Subrahmanian, V.S.: Maintaining views incrementally. In: SIGMOD Conference, pp. 157–166. ACM Press (1993)
36. Hai, R., Koutras, C., Quix, C., Jarke, M.: Data lakes: A survey of functions and systems. *IEEE Trans. Knowl. Data Eng.* **35**(12), 12,571–12,590 (2023)
37. Halevy, A.Y., Franklin, M.J., Maier, D.: Principles of dataspace systems. In: PODS, pp. 1–9. ACM (2006)
38. Hong, S., Lin, Y., Liu, B., Liu, B., Wu, B., Zhang, C., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhang, L., Yang, M., Zhuge, M., Guo, T., Zhou, T., Tao, W., Tang, R., Lu, X., Zheng, X., Liang, X., Fei, Y., Cheng, Y., Ni, Y., Gou, Z., Xu, Z., Luo, Y., Wu, C.: Data interpreter: An LLM agent for data science. In: ACL (Findings), *Findings of ACL*, vol. ACL 2025, pp. 19,796–19,821. Association for Computational Linguistics (2025)
39. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., Schmidhuber, J.: Metagpt: Meta programming for A multi-agent collaborative framework. In: ICLR. OpenReview.net (2024)
40. Hou, X., Zhao, Y., Wang, S., Wang, H.: Model context protocol (MCP): landscape, security threats, and future research directions. *CoRR* **abs/2503.23278** (2025)
41. Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K.: Guide to attribute based access control (ABAC) definition and considerations. NIST Special Publication 800-162, National Institute of Standards and Technology, Gaithersburg, MD (2014). DOI 10.6028/NIST.SP.800-162. URL <https://doi.org/10.6028/NIST.SP.800-162>
42. Isard, M., Budiu, M., Yu, Y., Birrell, A., Fetterly, D.: Dryad: distributed data-parallel programs from sequential building blocks. In: EuroSys, pp. 59–72. ACM (2007)
43. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with gpus. *IEEE Trans. Big Data* **7**(3), 535–547 (2021)
44. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.: Dense passage retrieval for open-domain question answering. In: EMNLP (1), pp. 6769–6781. Association for Computational Linguistics (2020)
45. Khatiwada, A., Fan, G., Shraga, R., Chen, Z., Gatterbauer, W., Miller, R.J., Riedewald, M.: SANTOS: relationship-based semantic table union search. *Proc. ACM Manag. Data* **1**(1), 9:1–9:25 (2023)
46. Kiela, D., Bartolo, M., Nie, Y., Kaushik, D., Geiger, A., Wu, Z., Vidgen, B., Prasad, G., Singh, A., Ringshia, P., Ma, Z., Thrush, T., Riedel, S., Waseem, Z., Stenetorp, P., Jia, R., Bansal, M., Potts, C., Williams, A.: Dynabench: Rethinking benchmarking in NLP. In: NAACL-HLT, pp. 4110–4124. Association for Computational Linguistics (2021)
47. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M.W., Keutzer, K., Gholami, A.: An LLM compiler for parallel function calling. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 24,370–24,391. PMLR / OpenReview.net (2024)
48. Kolev, B., Valduriez, P., Bondiombouy, C., Jiménez-Peris, R., Pau, R., Pereira, J.: Cloudmdsql: querying heterogeneous cloud data stores with a common language. *Distributed Parallel Databases* **34**(4), 463–503 (2016)
49. Kreps, J.: The log: What every software engineer should know about real-time data’s unifying abstraction (2013). URL <https://www.linkedin.com/blog/engineering/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>
50. Kriuk, B., Ng, L.: Q-kvcomm: Efficient multi-agent communication via adaptive KV cache compression. *CoRR* **abs/2512.17914** (2025)
51. Kumari, L., Wang, S., Zhou, T., Sarda, N., Rowe, A., Bilmes, J.: BumbleBee: Dynamic KV-cache streaming submodular summarization for infinite-context transformers. In: First Conference on Language Modeling (COLM) (2024). URL <https://openreview.net/pdf?id=8wORApM5yG>
52. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: SOSp, pp. 611–626. ACM (2023)
53. Lee, A.W., Chan, J., Fu, M., Kim, N., Mehta, A., Raghavan, D., Çetintemel, U.: Semantic integrity constraints: Declarative guardrails for ai-augmented data processing systems. *Proc. VLDB Endow.* **18**(11), 4073–4080 (2025)
54. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: NeurIPS (2020)
55. Li, G., Hammoud, H., Itani, H., Khizbullin, D., Ghanem, B.: CAMEL: communicative agents for

- "mind" exploration of large language model society. In: NeurIPS (2023)
56. Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., Li, Y.: Api-bank: A comprehensive benchmark for tool-augmented llms. In: EMNLP, pp. 3102–3116. Association for Computational Linguistics (2023)
 57. Li, Z., Zhang, H., Wei, C., Lu, P., Nie, P., Lu, Y., Bai, Y., Feng, S., Zhu, H., Zhong, M., Zhang, Y., Xie, J., Choi, Y., Zou, J., Han, J., Chen, W., Lin, J., Jiang, D., Zhang, Y.: Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction. CoRR **abs/2605.05242** (2026)
 58. Liu, C., Russo, M., Cafarella, M.J., Cao, L., Chen, P.B., Chen, Z., Franklin, M.J., Kraska, T., Madden, S., Shahout, R., Vitagliano, G.: Palimpsest: Optimizing ai-powered analytics with declarative query processing. In: CIDR. www.cidrdb.org (2025)
 59. Liu, S., Ponnappalli, S., Shankar, S., Zeighami, S., Zhu, A., Agarwal, S., Chen, R., Suwito, S., Yuan, S., Stolica, I., Zaharia, M., Cheung, A., Crooks, N., Gonzalez, J., Parameswaran, A.G.: Supporting our AI overlords: Redesigning data systems to be agent-first. In: CIDR. www.cidrdb.org (2026)
 60. Liu, Z., Hu, H., Zhang, S., Guo, H., Ke, S., Liu, B., Wang, Z.: Reason for future, act for now: A principled architecture for autonomous LLM agents. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 31,186–31,261. PMLR / OpenReview.net (2024)
 61. Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, H., Ma, S., Ma, S., Li, M., Yin, G., Wang, Z., Pang, R.: Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In: NAACL (Findings), *Findings of ACL*, vol. NAACL 2025, pp. 1160–1183. Association for Computational Linguistics (2025)
 62. Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J.E., Stolica, I.: Agentix: An efficient serving engine for LLM agents as general programs. In: NSDI, pp. 2443–2459. USENIX Association (2026)
 63. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-refine: Iterative refinement with self-feedback. In: NeurIPS (2023)
 64. Marcus, R., Negi, P., Mao, H., Tatbul, N., Alizadeh, M., Kraska, T.: Bao: Making learned query optimization practical. In: SIGMOD Conference, pp. 1275–1288. ACM (2021)
 65. Mazumder, M., Banbury, C.R., Yao, X., Karlas, B., Rojas, W.G., Damos, S.F., Damos, G., He, L., Parrish, A., Kirk, H.R., Quaye, J., Rastogi, C., Kiela, D., Jurado, D., Kanter, D., Mosquera, R., Cukierski, W., Ciro, J., Aroyo, L., Acun, B., Chen, L., Raje, M., Bartolo, M., Eyuboglu, E.S., Ghorbani, A., Goodman, E.D., Howard, A., Inel, O., Kane, T., Kirkpatrick, C.R., Sculley, D., Kuo, T., Mueller, J.W., Thrush, T., Vanschoren, J., Warren, M., Williams, A., Yeung, S., Ardalani, N., Paritosh, P.K., Zhang, C., Zou, J.Y., Wu, C., Coleman, C., Ng, A.Y., Mattson, P., Reddi, V.J.: Dataperf: Benchmarks for data-centric AI development. In: NeurIPS (2023)
 66. Mekala, D., Weston, J., Lanchantin, J., Raileanu, R., Lomeli, M., Shang, J., Dwivedi-Yu, J.: TOOLVERIFIER: generalization to new tools via self-verification. In: EMNLP (Findings), *Findings of ACL*, vol. EMNLP 2024, pp. 5026–5041. Association for Computational Linguistics (2024)
 67. Microsoft: Ask copilot questions about your data - power bi (2026). URL <https://learn.microsoft.com/en-us/power-bi/create-reports/copilot-ask-data-question>
 68. Microsoft: Semantic indexing for Microsoft 365 Copilot (2026). URL <https://learn.microsoft.com/en-us/microsoftsearch/semantic-index-for-copilot>
 69. Missier, P., Belhajjame, K., Cheney, J.: The W3C PROV family of specifications for modelling provenance meta-data. In: EDBT, pp. 773–776. ACM (2013)
 70. Model Context Protocol: Model Context Protocol specification (2025). URL <https://modelcontextprotocol.io/specification/2025-06-18>
 71. Mohan, C., Haderle, D., Lindsay, B.G., Pirahesh, H., Schwarz, P.M.: ARIES: A transaction recovery method supporting fine-granularity locking and partial roll-backs using write-ahead logging. ACM Trans. Database Syst. **17**(1), 94–162 (1992)
 72. MongoDB: MongoDB manual (2026). URL <https://www.mongodb.com/docs/manual/>
 73. Narechania, A., Srinivasan, A., Stasko, J.T.: NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. IEEE Trans. Vis. Comput. Graph. **27**(2), 369–379 (2021)
 74. Nargesian, F., Zhu, E., Miller, R.J., Pu, K.Q., Arocena, P.C.: Data lake management: Challenges and opportunities. Proc. VLDB Endow. **12**(12), 1986–1989 (2019)
 75. Neumann, T., Weikum, G.: RDF-3X: a risc-style engine for RDF. Proc. VLDB Endow. **1**(1), 647–659 (2008)
 76. OpenAI: Introducing deep research (2025). URL <https://openai.com/index/introducing-deep-research/>
 77. OpenAI: Codex CLI (2026). URL <https://learn.chatgpt.com/docs/codex/cli>
 78. OpenAI: Codex Cloud (2026). URL <https://learn.chatgpt.com/docs/cloud>

79. Oussous, A., Benjelloun, F., Lahcen, A.A., Belfkih, S.: Big data technologies: A survey. *J. King Saud Univ. Comput. Inf. Sci.* **30**(4), 431–448 (2018)
80. Packer, C., Fang, V., Patil, S.G., Lin, K., Wooders, S., Gonzalez, J.E.: Memgpt: Towards llms as operating systems. *CoRR* **abs/2310.08560** (2023)
81. Pan, J.J., Wang, J., Li, G.: Survey of vector database management systems. *VLDB J.* **33**(5), 1591–1615 (2024)
82. Pan, Z., Patel, A., Shen, Y., Hu, Z., Guan, Y., Li, W., Qin, L., Wang, Y., Ding, Y.: Kvflow: Efficient prefix caching for accelerating llm-based multi-agent workflows. In: *NeurIPS* (2025)
83. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative agents: Interactive simulators of human behavior. In: *UIST*, pp. 2:1–2:22. *ACM* (2023)
84. Patel, L., Jha, S., Pan, M., Gupta, H., Asawa, P., Guestrin, C., Zaharia, M.: Semantic operators: A declarative model for rich, ai-based data processing (2025). URL <https://arxiv.org/abs/2407.11418>
85. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. In: *NeurIPS* (2024)
86. PostgreSQL: PostgreSQL (2026). URL <https://www.postgresql.org>
87. Qi, K., Xie, D., Li, W., Zhang, H., Zhu, Y., Yu, J.X., Zhao, K.: Sema: A high-performance system for llm-based semantic query processing. *CoRR* **abs/2603.11622** (2026)
88. Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M.: Chatdev: Communicative agents for software development. In: *ACL* (1), pp. 15,174–15,186. Association for Computational Linguistics (2024)
89. Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., Sun, M.: Scaling large language model-based multi-agent collaboration. In: *ICLR. OpenReview.net* (2025)
90. Qiao, S., Gui, H., Lv, C., Jia, Q., Chen, H., Zhang, N.: Making language models better tool learners with execution feedback. In: *NAACL-HLT*, pp. 3550–3568. Association for Computational Linguistics (2024)
91. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M.: Toolllm: Facilitating large language models to master 16000+ real-world apis. In: *ICLR. OpenReview.net* (2024)
92. Raasveldt, M., Mühleisen, H.: Duckdb: an embeddable analytical database. In: *SIGMOD Conference*, pp. 1981–1984. *ACM* (2019)
93. Rezazadeh, A., Li, Z., Lou, A., Zhao, Y., Wei, W., Bao, Y.: Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control. *CoRR* **abs/2505.18279** (2025)
94. Robertson, S.E., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. *Found. Trends Inf. Retr.* **3**(4), 333–389 (2009)
95. Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T.: Identifying the risks of LM agents with an lm-emulated sandbox. In: *ICLR. OpenReview.net* (2024)
96. Russo, M., Liu, C., Sudhir, S., Vitagliano, G., Cafarella, M.J., Kraska, T., Madden, S.: Abacus: A cost-based optimizer for semantic operator systems. *Proc. VLDB Endow.* **19**(5), 1060–1073 (2026)
97. Salemi, A., Parmar, M., Goyal, P., Song, Y., Yoon, J., Zamani, H., Palangi, H., Pfister, T.: Llm-based multi-agent blackboard system for information discovery in data science. *CoRR* **abs/2510.01285** (2025)
98. Salemi, A., Zeng, C., Nijasure, A., Chung, J., Rahimi, R., Diaz, F., Zamani, H.: Grepseek: Training search agents for direct corpus interaction. *CoRR* **abs/2605.29307** (2026)
99. Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E.: Role-based access control models. *Computer* **29**(2), 38–47 (1996)
100. Santhanam, K., Raghavan, D., Rahman, M.S., Venkatesh, T., Kunjal, N., Thaker, P., Levis, P.A., Zaharia, M.: ALTO: an efficient network orchestrator for compound AI systems. In: *EuroMLSys@EuroSys*, pp. 117–125. *ACM* (2024)
101. Satriani, D., Veltri, E., Santoro, D., Rosato, S., Varriale, S., Papotti, P.: Logical and physical optimizations for SQL query execution over large language models. *Proc. ACM Manag. Data* **3**(3), 181:1–181:28 (2025)
102. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: *NeurIPS* (2023)
103. Shang, Y., Li, Y., Zhao, K., Ma, L., Liu, J., Xu, F., Li, Y.: Agentsquare: Automatic LLM agent search in modular design space. In: *ICLR. OpenReview.net* (2025)
104. Shankar, S., Chambers, T., Shah, T., Parameswaran, A.G., Wu, E.: Docetl: Agentic query rewriting and evaluation for complex document processing. *Proc. VLDB Endow.* **18**(9), 3035–3048 (2025)
105. Shankar, S., Zeighami, S., Parameswaran, A.G.: Task cascades for efficient unstructured data processing. *CoRR* **abs/2601.05536** (2026)
106. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: Hugginggpt: Solving AI tasks with chatgpt and its friends in hugging face. In: *NeurIPS* (2023)

107. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: language agents with verbal reinforcement learning. In: NeurIPS (2023)
108. Shute, J., Vingralek, R., Samwel, B., Handy, B., Whipkey, C., Rollins, E., Oancea, M., Littlefield, K., Menestrina, D., Ellner, S., Cieslewicz, J., Rae, I., Stancescu, T., Apte, H.: F1: A distributed SQL database that scales. *Proc. VLDB Endow.* **6**(11), 1068–1079 (2013)
109. Snowflake: Cortex Analyst (2026). URL <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst>
110. Srinivas, K., Dolby, J., Abdelaziz, I., Hassanzadeh, O., Kokel, H., Khatiwada, A., Pedapati, T., Chaudhury, S., Samulowitz, H.: Lakebench: Benchmarks for data discovery over data lakes. *CoRR* **abs/2307.04217** (2023)
111. Stonebraker, M., Brown, P., Zhang, D., Becla, J.: Scidb: A database management system for applications with complex analytics. *Comput. Sci. Eng.* **15**(3), 54–62 (2013)
112. Sun, Q., Luo, Y., Zhang, W., Li, S., Li, J., Niu, K., Kong, X., Liu, W.: Docs2kg: Unified knowledge graph construction from heterogeneous documents assisted by large language models. *CoRR* **abs/2406.02962** (2024)
113. Wadlom, N., Shen, J., Lu, Y.: Efficient LLM serving for agentic workflows: A data systems perspective. *CoRR* **abs/2603.16104** (2026)
114. Wang, J., Feng, J.: Unify: An unstructured data analytics system. In: ICDE, pp. 4662–4674. IEEE (2025)
115. Wang, J., Li, G.: AOP: automated and interactive LLM pipeline orchestration for answering complex queries. In: CIDR. www.cidrdb.org (2025)
116. Wang, J., Ma, Z., Li, Y., Zhang, S., Chen, C., Chen, K., Le, X.: GTA: A benchmark for general tool agents. *CoRR* **abs/2407.08713** (2024)
117. Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., Yu, K., Yuan, Y., Zou, Y., Long, J., Cai, Y., Li, Z., Zhang, Z., Mo, Y., Gu, J., Jiang, R., Wei, Y., Xie, C.: Milvus: A purpose-built vector data management system. In: SIGMOD Conference, pp. 2614–2627. ACM (2021)
118. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W.X., Wei, Z., Wen, J.: A survey on large language model based autonomous agents. *Frontiers Comput. Sci.* **18**(6), 186,345 (2024)
119. Wang, X., Li, B., Song, Y., Xu, F.F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H.H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., et al.: Openhands: An open platform for AI software developers as generalist agents. In: ICLR. OpenReview.net (2025)
120. Wang, Z.Z., Mao, J., Fried, D., Neubig, G.: Agent workflow memory. In: ICML, *Proceedings of Machine Learning Research*, vol. 267. PMLR / OpenReview.net (2025)
121. Weng, L., Liu, D., Zhu, R., Ding, B., Zhou, J.: Bridgescope: A universal toolkit for bridging large language models and databases. In: CIDR. www.cidrdb.org (2026)
122. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., Wang, C.: Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. *CoRR* **abs/2308.08155** (2023)
123. Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T.J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., Yu, T.: Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In: NeurIPS (2024)
124. Xie, Z., Kang, H., Sheng, Y., Krishna, T., Fatahalian, K., Kozyrakis, C.: AI metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution. In: MLSys. OpenReview.net/mlsys.org (2025)
125. Xu, B., Peng, Z., Lei, B., Mukherjee, S., Liu, Y., Xu, D.: Rewoo: Decoupling reasoning from observations for efficient augmented language models. *CoRR* **abs/2305.18323** (2023)
126. Xu, C., McAuley, J.J., Wang, P.: Mirror: A natural language interface for data querying, summarization, and visualization. In: WWW (Companion Volume), pp. 49–52. ACM (2023)
127. Xue, S., Jiang, C., Shi, W., Cheng, F., Chen, K., Yang, H., Zhang, Z., He, J., Zhang, H., Wei, G., Zhao, W., Zhou, F., Qi, D., Yi, H., Liu, S., Chen, F.: DB-GPT: empowering database interactions with private large language models. *CoRR* **abs/2312.17449** (2023)
128. Yang, F., Tschetter, E., Léauté, X., Ray, N., Merlino, G., Ganguli, D.: Druid: a real-time analytical data store. In: SIGMOD Conference, pp. 157–168. ACM (2014)
129. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: Swe-agent: Agent-computer interfaces enable automated software engineering. In: NeurIPS (2024)
130. Yao, S., Shinn, N., Razavi, P., Narasimhan, K.R.: τ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In: ICLR. OpenReview.net (2025)
131. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: ICLR. OpenReview.net (2023)
132. Ye, H., Gao, Z., Ma, M., Wang, Q., Fu, Y., Chung, M., Lin, Y., Liu, Z., Zhang, J., Zhuo, D., Chen, Y.: KV-COMM: online cross-context kv-cache communication for efficient llm-based multi-agent systems. *CoRR*

- abs/2510.12872** (2025)
133. Ye, J., Li, S., Li, G., Huang, C., Gao, S., Wu, Y., Zhang, Q., Gui, T., Huang, X.: Toolsworld: Unveiling safety issues of large language models in tool learning across three stages. In: ACL (1), pp. 2181–2211. Association for Computational Linguistics (2024)
 134. Ye, J., Wu, Y., Gao, S., Huang, C., Li, S., Li, G., Fan, X., Zhang, Q., Gui, T., Huang, X.: Rotbench: A multi-level benchmark for evaluating the robustness of large language models in tool learning. In: EMNLP, pp. 313–333. Association for Computational Linguistics (2024)
 135. Yu, G., Jeong, J.S., Kim, G., Kim, S., Chun, B.: Orca: A distributed serving system for transformer-based generative models. In: OSDI, pp. 521–538. USENIX Association (2022)
 136. Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauly, M., Franklin, M.J., Shenker, S., Stoica, I.: Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In: NSDI, pp. 15–28. USENIX Association (2012)
 137. Zeighami, S., Lin, Y., Shankar, S., Parameswaran, A.G.: Llm-powered proactive data systems. IEEE Data Eng. Bull. **49**(1), 90–103 (2025)
 138. Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., Yan, S.: G-memory: Tracing hierarchical memory for multi-agent systems. CoRR **abs/2506.07398** (2025)
 139. Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J.X., Chen, T.: Cut the crap: An economical communication pipeline for llm-based multi-agent systems. In: ICLR. OpenReview.net (2025)
 140. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: OSDI, pp. 1187–1204. USENIX Association (2020)
 141. Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., Wu, C.: Aflow: Automating agentic workflow generation. In: ICLR. OpenReview.net (2025)
 142. Zhang, Q., Wornow, M., Olukotun, K.: Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. In: NeurIPS (2025)
 143. Zheng, Y., Li, P., Liu, W., Liu, Y., Luan, J., Wang, B.: Toolrerank: Adaptive and hierarchy-aware reranking for tool retrieval. In: LREC/COLING, pp. 16,263–16,273. ELRA and ICCL (2024)
 144. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI, pp. 193–210. USENIX Association (2024)
 145. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., Wang, Y.: Language agent tree search unifies reasoning, acting, and planning in language models. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 62,138–62,160. PMLR / OpenReview.net (2024)
 146. Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G.: Webarena: A realistic web environment for building autonomous agents. In: ICLR. OpenReview.net (2024)
 147. Zhu, Y., Wang, L., Yang, C., Lin, X., Li, B., Zhou, W., Liu, X., Peng, Z., Luo, T., Li, Y., Chai, C., Chen, C., Di, S., Fan, J., Sun, J., Tang, N., Tsung, F., Wang, J., Wu, C., Xu, Y., Zhang, S., Zhang, Y., Zhou, X., Li, G., Luo, Y.: A survey of data agents: Emerging paradigm or overstated hype? CoRR **abs/2510.23587** (2025)
 148. Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., Schmidhuber, J.: Gptswarm: Language agents as optimizable graphs. In: ICML, *Proceedings of Machine Learning Research*, vol. 235, pp. 62,743–62,767. PMLR / OpenReview.net (2024)
 149. Zou, L., Mo, J., Chen, L., Özsu, M.T., Zhao, D.: gstore: Answering SPARQL queries via subgraph matching. Proc. VLDB Endow. **4**(8), 482–493 (2011)